

High-Level System Design

HLD Interview Guide

100 Questions & Answers — SDE2 Interview Prep

Level: Mid (2–4 yrs)	Stack: Distributed Systems	Roles: SDE2 / Backend
----------------------	----------------------------	-----------------------

#	Topic	Questions
1	Fundamentals & Estimation	Q1–Q10
2	Databases & Storage	Q11–Q22
3	Caching & CDN	Q23–Q32
4	Messaging & Queues	Q33–Q42
5	Distributed Systems Concepts	Q43–Q55
6	Microservices & APIs	Q56–Q65
7	Observability & Reliability	Q66–Q73
8	Classic System Designs	Q74–Q83
9	Advanced System Designs	Q84–Q92
10	Real-World Trade-offs	Q93–Q100

100 Questions & Answers with Diagrams and Trade-offs

> **Prepared for:** SDE2 role interviews | **Focus:** Distributed Systems, Scalability, Architecture | **Level:** Mid-level (2–4 years)

Table of Contents

[Fundamentals & Estimation](#1-fundamentals--estimation) — Q1–Q12
[Databases & Storage](#2-databases--storage) — Q13–Q25
[Caching & CDN](#3-caching--cdn) — Q26–Q34
[Messaging & Queues](#4-messaging--queues) — Q35–Q43
[Distributed Systems Concepts](#5-distributed-systems-concepts) — Q44–Q56
[Microservices & APIs](#6-microservices--apis) — Q57–Q65
[Observability & Reliability](#7-observability--reliability) — Q66–Q73
[Classic System Designs](#8-classic-system-designs) — Q74–Q87
[Advanced System Designs](#9-advanced-system-designs) — Q88–Q95
[Real-World Trade-offs](#10-real-world-trade-offs) — Q96–Q100

Fundamentals & Estimation

Q1. How do you approach a system design interview?

Interview
framework

Easy

Use the RESHADED framework: Requirements → Estimation → Schema → High-level design → APIs → Deep dive → Evaluate.

Step 1 – Requirements (5 min)

Functional: What does it do? (core features only)
Non-functional: Scale, latency, availability, consistency
Out of scope: What won't you design?

Step 2 – Estimation (3 min)

DAU, QPS, storage, bandwidth

Step 3 – Data Model / Schema (3 min)

Key entities, relationships, access patterns

Step 4 – High-Level Design (10 min)

Draw boxes: Client → LB → API servers → Cache → DB
Identify bottlenecks

Step 5 – API Design (3 min)

Core endpoints (3-4 max)

Step 6 – Deep Dive (15 min)

Pick 2 hardest components – drill down

Step 7 – Evaluate / Trade-offs (2 min)

What breaks at 10x scale? What would you change?

Interview tip: "Always clarify before designing. Spending 5 minutes on requirements prevents 20 minutes of wasted design. Ask: 'Should I focus on read scale or write scale?'"

Q2. How do you estimate requests per second (QPS)?

Back-of-envelope

Easy

Twitter-like system:

DAU = 100 million users

Each user: 5 tweets/day read, 0.1 tweets/day write

Read QPS = $100M \times 5 / 86400 \approx 5,800$ reads/sec $\rightarrow \sim 6K$ reads/secWrite QPS = $100M \times 0.1 / 86400 \approx 115$ writes/sec $\rightarrow \sim 120$ writes/secPeak QPS = avg $\times 3 \approx 18K$ reads/secRule of thumb: 1 day = 86,400 sec $\approx 100K$ secSimplification: $100M / 100K = 1,000$ QPS per action/day

Key numbers to memorize:

1 million users, 1 req/day $\rightarrow \sim 12$ QPS10 million users, 10 req/day $\rightarrow \sim 1,200$ QPS100 million DAU, 1 req/day $\rightarrow \sim 1,200$ QPS1 billion DAU, 10 req/day $\rightarrow \sim 120,000$ QPS***Interview tip:** "Round aggressively. 86,400 $\rightarrow$ 100,000. 1,157 $\rightarrow$ $\sim 1,200$. Interviewers care about order of magnitude, not exact numbers."***Q3. How do you estimate storage requirements?**

Storage estimation

Easy

Instagram-like system:

DAU = 500 million

Photos uploaded per day = 100 million

Average photo size = 200 KB

Daily storage = $100M \times 200$ KB = 20 TB/day5-year storage = 20 TB $\times 365 \times 5 = 36.5$ PBWith replication (3 $\times$) = ~ 110 PB

Storage for metadata:

Per photo = 500 bytes (URL, userID, timestamp, caption)

Daily metadata = $100M \times 500B = 50$ GB/day5-year metadata = ~ 90 TB

Size reference:

char = 1 byte

UUID = 16 bytes

int64 = 8 bytes

timestamp = 8 bytes

URL = ~ 100 bytestweet = ~ 300 bytes

photo = 100 KB – 5 MB

HD video = 1–2 GB/hour

Interview tip:** "Storage is cheap; mention tiered storage: hot (SSD, recent 30 days), warm (HDD, last year), cold (S3 Glacier, older). Dramatically reduces cost."Q4. How do you estimate bandwidth?**

Bandwidth estimation

Easy

YouTube-like system:

DAU = 1 billion

Each user watches: 30 min/day

Consistent Hash — minimal remapping when servers added/removed

Layer 4 (Transport): routes by IP/TCP — fast, no content inspection

Layer 7 (Application): routes by HTTP headers, URL, cookies — flexible

Health checks: LB polls /health every 10s, removes unhealthy servers

Interview tip: "Always mention health checks. A LB without health checks routes to dead servers. Ask about session stickiness — cookie-based is better than IP hash (mobile IPs change)."

Q7. What is a reverse proxy and how does it differ from a load balancer?

Proxy
concepts

Easy

Reverse Proxy sits in front of servers, acts on their behalf:

- SSL termination (decrypts HTTPS once, forwards HTTP internally)
- Caching (serves static content directly)
- Compression (gzip responses)
- Request routing
- Rate limiting
- Security (hide internal topology)

Load Balancer is a specialised reverse proxy focused on traffic distribution.

```
Client → [Reverse Proxy / LB] → Server 1
                                   → Server 2
                                   → Server 3
```

Tools:

Nginx: Reverse proxy + LB + static serving

HAProxy: High-performance TCP/HTTP LB

Envoy: Service mesh proxy (L7, observability built-in)

AWS ALB/NLB: Managed cloud LBs

Cloudflare: Global reverse proxy + CDN

Interview tip: "In production: Nginx or Envoy at the edge (SSL termination, routing), then an internal LB (ALB) for service-to-service. Don't conflate the two."

Q8. What is a CDN and when do you use one?

CDN
fundamentals

Easy

CDN (Content Delivery Network): geographically distributed cache servers (PoPs) serving content from the nearest location to the user.

When to use:

- Static assets (images, videos, JS, CSS)
- High-bandwidth content (video streaming)
- Global user base (latency reduction)
- DDoS protection (absorb traffic at edge)

How it works:

1. User requests `https://cdn.example.com/image.jpg`
2. DNS resolves to nearest CDN PoP (anycast or GeoDNS)
3. PoP checks cache — hit? → serve immediately (10ms)
4. Miss? → PoP fetches from origin, caches, serves (200ms)

Cache strategies:

Pull CDN: CDN fetches from origin on first miss (lazy)

Push CDN: You upload content to CDN proactively

Providers: Cloudflare, AWS CloudFront, Fastly, Akamai

Interview tip: "Always add CDN in your design for any user-facing media. It reduces origin load by 90%+ for static content. Mention cache-control headers for invalidation."

Medium

Interview tip: "DNS is not instant failover. With TTL=300s, clients may hit dead servers for 5 minutes. Use health-check-aware DNS (Route53 with health checks) + low TTL for critical services."

Easy

Eliminates TCP head-of-line blocking
Faster connection setup (0-RTT)

Interview tip: "For real-time systems (live video, gaming): UDP. For anything where data must arrive intact (payment, file upload): TCP. QUIC is becoming the standard for web traffic."

Q11. What is latency vs throughput vs availability?

System
metrics

Easy

Latency: time to serve ONE request

p50: median (50th percentile)

p99: 99th percentile – worst 1% of requests

p999: 99.9th percentile – worst 0.1% of requests

Always optimise p99, not just average

Targets: <10ms (excellent), <100ms (good), <1s (acceptable)

Throughput: requests served per unit time (QPS, TPS)

Throughput ↑ as concurrency ↑ (up to bottleneck)

Throughput ↓ when latency spikes

Availability: % of time system is operational

99% = 3.65 days downtime/year

99.9% = 8.77 hours downtime/year (3 nines)

99.99% = 52.6 min downtime/year (4 nines)

99.999% = 5.26 min downtime/year (5 nines)

SLA = contractual commitment to availability

Compound availability:

Two services A (99.9%) + B (99.9%) in series:

= $99.9\% \times 99.9\% = 99.8\%$

Each dependency reduces overall availability!

Interview tip: "When someone says 'low latency', ask: 'What's your p99 target?' Average latency hides long tail. A system with avg 10ms but p99 500ms is still broken for 1% of users."

Q12. How do you calculate the number of servers needed?

Capacity
planning

Easy

Given:

Peak QPS = 50,000 requests/sec

Avg request duration = 50ms

One server handles = 1000 req/sec (rule of thumb for web server)

Calculation:

Servers needed = Peak QPS / QPS per server

= $50,000 / 1,000 = 50$ servers

With 30% headroom: $50 / 0.7 = \sim 72$ servers

For CPU-bound work:

Concurrency per server = CPU cores $\times$ (1 / CPU utilisation target)

16-core server at 70% CPU = ~ 22 concurrent requests

At 50ms latency: $22 / 0.05s = 440$ QPS/server

For I/O-bound work (waiting on DB):

Concurrency = threads or goroutines waiting

Goroutines: 10,000 goroutines/server at 50ms = 200,000 QPS/server

(Go is extremely efficient for I/O-bound work)

DB connections:

Connection pool per server = 10-25 connections

Total DB connections = 72 servers × 15 = 1,080 → needs connection pooler (PgBouncer)

Interview tip: "In Go specifically: goroutine-based servers can handle far more concurrent connections than thread-per-request servers (Java/Python). Mention this as an advantage."

Databases & Storage

Q13. When do you choose SQL vs NoSQL?

Database
selection

Medium

Choose SQL (PostgreSQL, MySQL) when:

- ACID transactions required (payments, inventory)
- Complex queries (JOINS, aggregations)
- Schema is well-defined and stable
- Data integrity constraints (foreign keys)
- Moderate scale (<10M rows/table without sharding)

Choose NoSQL when:

- Schema-less or frequently changing schema
- Massive scale (billions of records)
- Simple access patterns (key-value, document by ID)
- High write throughput
- Horizontal scaling is required

NoSQL categories:

Key-Value: Redis, DynamoDB – O(1) lookup by key

Document: MongoDB, Firestore – JSON documents, flexible schema

Wide-Column: Cassandra, HBase – high write throughput, time-series

Graph: Neo4j, Amazon Neptune – social graphs, fraud detection

Example trade-offs:

User profiles → MongoDB (flexible schema, read by ID)

Payments → PostgreSQL (ACID, joins, audit)

Session store → Redis (TTL, fast lookup)

Activity feed → Cassandra (time-series, append-heavy)

Social graph → Neo4j (friend-of-friend queries)

Interview tip: "Don't say 'NoSQL scales better.' Say: 'NoSQL sacrifices consistency and query flexibility for horizontal scale. For our access pattern (key lookup by user ID at 1M QPS), DynamoDB is appropriate.'"

Q14. What is database indexing and how does it work?

Indexing

Medium

Index: auxiliary data structure that speeds up lookups.

Trade-off: faster reads, slower writes (index must be updated).

B-Tree Index (default in PostgreSQL, MySQL):

- Balanced tree, O(log N) lookup, range scans
- Good for: =, <, >, BETWEEN, ORDER BY
- Size: roughly 10-30% of table size

Hash Index:

- O(1) lookup, no range scans
- Good for: = only

Composite Index: (col_a, col_b)

- Prefix rule: usable for col_a alone, col_a+col_b, NOT col_b alone
- Order matters: put high-cardinality columns first

Covering Index: index includes all columns needed for query

- SELECT name FROM users WHERE email=?
- Index on (email, name) → no table fetch needed

Partial Index: index on subset of rows

- CREATE INDEX ON orders(user_id) WHERE status='active'
- Smaller index, faster for filtered queries

Full-Text Index: inverted index for text search

- PostgreSQL: tsvector / tsquery
- Good for: LIKE '%term%' alternatives

Interview tip: "EXPLAIN ANALYZE in PostgreSQL shows query plan and actual execution time. Always run it before claiming a query is slow. Seq scan = missing index. Index scan = good."

Q15. What is database replication and what are its types?

Replication

Medium

Replication: copying data to multiple nodes for redundancy and read scaling.

Single-Leader (Master-Replica):

- One writable primary → changes replicated to read replicas
- Pros: simple, read scaling, failover
- Cons: write bottleneck at primary, replication lag

Replication lag: replica may be seconds/minutes behind primary

Read-your-writes: after write to primary, read from primary

Multi-Leader:

- Multiple writable nodes, sync changes peer-to-peer
- Pros: write throughput, multi-datacenter
- Cons: write conflicts (last-write-wins, merge, CRDTs)
- Used by: CockroachDB, Google Docs (CRDT)

Leaderless (Dynamo-style):

- Writes go to W out of N nodes
- Reads from R out of N nodes
- Quorum: $W + R > N$ ensures overlap
- Used by: Cassandra, DynamoDB

Synchronous replication: primary waits for replica ACK

- Strong consistency, higher write latency

Asynchronous replication: primary returns before replica ACK

- Lower write latency, possible data loss on primary failure

PostgreSQL streaming replication: async by default, can be sync

Interview tip: "Always ask: 'Is replication lag acceptable?' For user profile reads: yes. For inventory (race condition on last item): no — read from primary or use synchronous replication."

Q16. What is database sharding?

Sharding

Hard

Sharding: horizontal partitioning — split data across multiple DB nodes.

Shard Key selection (most important decision):

- Good: high cardinality, even distribution, no hotspots
- Bad: low cardinality (country = 3 values), monotonically increasing (time-based → hotspot)

Sharding strategies:

- Range-based: shard by user_id range [0-1M, 1M-2M, ...]
- Pros: range queries easy

Cons: hotspots (latest data all on one shard)

Hash-based: $\text{shard} = \text{hash}(\text{user_id}) \% N$

Pros: even distribution

Cons: range queries require all shards, resharding hard

Directory-based: lookup table maps keys $\rightarrow$ shards

Pros: flexible, easy resharding

Cons: lookup table is a bottleneck/SPOF

Consistent Hashing: virtual nodes on a ring

Pros: minimal data movement when adding/removing shards

Cons: more complex

Challenges:

Cross-shard joins: not supported $\rightarrow$ denormalise or application-level join

Cross-shard transactions: use 2-phase commit or saga pattern

Resharding: double-write during migration, painful

Hot shards: add virtual nodes, split shard

Rule of thumb: start with 1 shard, shard when $>50M$ rows and tuning is exhausted

Interview tip: "Interviewers want to hear: shard key selection rationale, resharding strategy, and how you handle cross-shard queries. Most systems shard too early — say 'I'd exhaust vertical scaling first.'"

Q17. What is the CAP theorem?

Distributed
systems
theory

Medium

CAP Theorem: In a distributed system, you can guarantee at most 2 of:

C – Consistency: every read sees the most recent write

A – Availability: every request gets a response (not an error)

P – Partition Tolerance: system works despite network partitions

Network partitions ALWAYS happen in distributed systems.

Therefore real choice is: C vs A during a partition.

CP systems (consistency over availability):

During partition: reject requests rather than serve stale data

Examples: HBase, Zookeeper, Spanner, etcd, Redis (single-node)

Use: financial transactions, inventory

AP systems (availability over consistency):

During partition: serve possibly stale data

Examples: DynamoDB, Cassandra, CouchDB

Use: social feeds, user profiles, DNS

CA systems: impossible in distributed systems (can't tolerate partition)

Only applies to single-node or non-distributed systems

Modern nuance – PACELC extends CAP:

Even without partition (P):

Latency (L) vs Consistency (C) trade-off

Examples: DynamoDB (EL – low latency, eventual consistency)

Spanner (EC – high consistency, higher latency)

Interview tip: "CAP is often misunderstood. The right answer: 'All distributed systems must tolerate partitions (P). The real question is CP vs AP.' Then map your design to one."

Q18. What is ACID vs BASE?Consistency
models**Medium**

ACID (relational databases):

Atomicity: transaction is all-or-nothing
 Consistency: DB moves from one valid state to another
 Isolation: concurrent transactions don't interfere
 Durability: committed data persists (written to disk/WAL)

Cost: locks, 2-phase commit, WAL overhead → lower throughput

BASE (NoSQL, distributed):

Basically Available: system is available even with failures
 Soft state: state may change over time (even without input)
 Eventually Consistent: system will be consistent... eventually

Benefit: higher throughput, lower latency, better availability

Isolation Levels (SQL):

Read Uncommitted: see dirty reads (dangerous)
 Read Committed: no dirty reads (PostgreSQL default)
 Repeatable Read: no non-repeatable reads
 Serializable: transactions run as if sequential (safest, slowest)

Anomalies:

Dirty read: reading uncommitted writes
 Non-repeatable read: re-read row changes within transaction
 Phantom read: range query returns different rows
 Write skew: two txns read same data, make conflicting updates

Interview tip: "For payment systems: ACID with Serializable isolation. For social feed: BASE with eventual consistency. Know where your system falls on this spectrum."

Q19. What is a write-ahead log (WAL) and why is it important?

Durability

Medium

WAL: append-only log of every change before applying to data files.

Purpose: crash recovery – replay WAL to restore state after failure.

How it works:

1. Transaction begins
2. Changes written to WAL (sequential write = fast)
3. Client gets ACK – data is durable
4. DB flushes WAL to data files asynchronously (background)
5. On crash: replay WAL from last checkpoint

Benefits:

- Durability without immediate random writes to data files
- Sequential writes are 10-100× faster than random writes
- Replication: stream WAL to replicas (PostgreSQL streaming replication)
- Point-in-time recovery: replay WAL to any timestamp

WAL in different systems:

PostgreSQL: WAL (pg_wal/)
 MySQL/InnoDB: redo log
 Kafka: commit log (IS the WAL)
 RocksDB/LevelDB: Write-Ahead Log before memtable
 Redis AOF: Append-Only File (WAL-like persistence)

fsync: force OS buffer to disk

fsync=on: safe but slower (disk write per transaction)

fsync=off: dangerous – data loss on power failure

Interview tip: "PostgreSQL's WAL enables logical replication, streaming replication, and PITR. Understanding WAL shows you know databases beyond SQL syntax."

Q20. What is the difference between B-Tree and LSM-Tree storage engines?

Storage
engine
internals

Hard

B-Tree (PostgreSQL, MySQL InnoDB):

Structure: sorted balanced tree on disk
Write: find page, update in-place → random I/O
Read: $O(\log N)$ traversal → fast reads
Good for: read-heavy, random access workloads
Examples: PostgreSQL heap + B-tree index

LSM-Tree (Cassandra, RocksDB, LevelDB):

Structure: in-memory buffer (memtable) → SSTables on disk
Write: append to WAL + memtable → sequential I/O → flush to SSTable
Read: check memtable + each SSTable level (bloom filter helps)
Compaction: merge SSTables periodically (background I/O)
Good for: write-heavy workloads, time-series, event streams

	B-Tree	LSM-Tree
		
Writes	Random I/O (slow)	Sequential (fast)
Reads	Fast	Slower (multiple files)
Space	Less wasted	Write amplification
Compaction	None needed	Required (background)
Use case	OLTP	Write-heavy, analytics

Interview tip: "LSM trees are why Cassandra and RocksDB handle millions of writes/sec. The memtable absorbs all writes in RAM, then flushes sequentially. Random write → sequential write transformation."

Q21. What is connection pooling and why is it needed?

Connection
management

Easy

Problem: opening a DB connection is expensive (~5ms, ~50MB RAM).

Solution: maintain a pool of reusable connections.

Without pooling:

$100 \text{ req/sec} \times 5\text{ms connection time} = 500\text{ms wasted per second in connection overhead}$

With pooling (PgBouncer for PostgreSQL):

Pool of 20 persistent connections shared by 1000 goroutines
Connection reuse → ~0ms overhead after warmup

Parameters:

min_pool_size: connections always kept open (warm pool)
max_pool_size: maximum concurrent connections
connection_lifetime: recycle old connections (prevent stale)
idle_timeout: close unused connections

PgBouncer modes:

Session pooling: 1 DB conn per client session (least efficient)
Transaction pooling: 1 DB conn per transaction (most efficient, default)
Statement pooling: 1 DB conn per statement (limits prepared statements)

Rules of thumb:

Max connections per PostgreSQL: 100-500

With PgBouncer: 1000s of app connections → 100 DB connections
 $\text{app_servers} \times \text{threads_per_server} \times \text{max_pool} = \text{total DB connections}$

Go specifics:

```
database/sql has built-in pool
db.SetMaxOpenConns(25)
db.SetMaxIdleConns(10)
db.SetConnMaxLifetime(5 * time.Minute)
```

Interview tip: "Mention PgBouncer by name. Not having a connection pooler with PostgreSQL at scale is a common mistake. At 1000 connections, PostgreSQL itself becomes a bottleneck."

Q22. What is database denormalization and when do you use it?

Data
modelling

Medium

Normalization: eliminate redundancy, ensure data integrity (OLTP)

Denormalization: introduce redundancy to speed up reads (OLAP/high-scale)

Example – Twitter timeline:

Normalized:

```
tweets table: tweet_id, user_id, content, created_at
followers table: follower_id, followed_id
Query timeline: SELECT t.* FROM tweets t
                JOIN followers f ON t.user_id = f.followed_id
                WHERE f.follower_id = ? ORDER BY t.created_at DESC
Problem: JOIN across billions of rows = too slow
```

Denormalized (fan-out on write):

```
timeline table: user_id, tweet_id, content, author_name, author_avatar
Pre-compute timeline for each user on tweet creation
Query timeline: SELECT * FROM timeline WHERE user_id=? ORDER BY created_at DESC
Fast: single table scan with index
```

Trade-offs:

```
Denormalized reads: O(1) → blazing fast
Write amplification: 1 tweet × 10K followers = 10K writes
Storage: duplicated data
Consistency: must update all copies on profile change
```

When to denormalize:

- Read QPS >> Write QPS
- Join query is the bottleneck
- Data rarely changes (e.g., tweet content is immutable)

Interview tip: "Twitter's timeline is the canonical denormalization example. But don't denormalize everything — start normalised, then denormalize specific hot query paths."

Q23. What is the N+1 query problem and how do you fix it?

Query
optimisation

Medium

N+1 problem: fetching N records then querying 1 more time per record.

Example (Go + SQL):

```
posts, _ := db.Query("SELECT id FROM posts LIMIT 100") // 1 query
for _, post := range posts {
    author, _ := db.QueryRow("SELECT name FROM users WHERE id=?", post.UserID) // 100 queries!
}
// Total: 101 queries
```

Fix 1: JOIN

```
SELECT p.*, u.name FROM posts p
JOIN users u ON p.user_id = u.id
LIMIT 100
// 1 query
```

Fix 2: IN clause batching

```
userIDs := extractUserIDs(posts)
users, _ := db.Query("SELECT id, name FROM users WHERE id IN (?)", userIDs)
userMap := toMap(users)
for _, post := range posts { post.Author = userMap[post.UserID] }
// 2 queries
```

Fix 3: DataLoader (batch + cache, common in GraphQL)

Batch all user lookups within a request window
Cache results within request lifecycle

Fix 4: Preload / Eager loading (ORMs)

```
db.Preload("Author").Find(&posts) // GORM generates JOIN automatically
```

Interview tip: "N+1 is the most common performance bug in production systems. Detection: log queries with duration, alert on >10 queries for a single request. Fix during code review."

Q24. What is OLTP vs OLAP?

Workload
types

Easy

OLTP (Online Transaction Processing):

Many small transactions, real-time, read/write
Highly normalized, ACID, low latency
Example: e-commerce checkout, banking, social feed
Systems: PostgreSQL, MySQL, DynamoDB

OLAP (Online Analytical Processing):

Few large queries, batch/near-real-time, mostly read
Denormalized (star schema), column-oriented, high throughput
Example: business intelligence, reporting, dashboards
Systems: BigQuery, Redshift, Snowflake, ClickHouse

Column-oriented storage (OLAP):

Data stored column by column: [col1_all_rows], [col2_all_rows]
Query: SELECT SUM(revenue) FROM orders → reads ONLY revenue column
Compression: same data type → 10-100× compression
vs Row-oriented: reads entire row to access one column → wasteful

Lambda architecture:

Batch layer (Hadoop/Spark): accurate, slow (hours)
Speed layer (Kafka/Flink): fast, approximate (seconds)
Serving layer: merges both for queries

Kappa architecture: stream-only (Kafka as source of truth)

Simpler, stream everything, replay for reprocessing

Data warehouse pipeline:

```
OLTP DB → Kafka/CDC → ETL (dbt) → Data Warehouse → BI tools
```

Interview tip: "When someone asks 'how would you build an analytics dashboard?' — separate OLAP store from OLTP DB immediately. Never run analytics queries on your production OLTP database."

Q25. What is a distributed transaction and how do you handle it?

Distributed
transactions

Hard

Problem: updating data across multiple services/databases atomically.

Example: Transfer \$100: debit from Account Service + credit to Payment Service

Option 1: Two-Phase Commit (2PC)

Phase 1 (Prepare): coordinator asks all participants to prepare

Phase 2 (Commit): if all say yes → commit; else → rollback

Pros: strong consistency

Cons: blocking, coordinator is SPOF, latency, slow

Option 2: Saga Pattern (preferred for microservices)

Break transaction into local transactions + compensating transactions

Each service does its part; on failure, run compensating actions

Choreography saga (event-driven):

OrderService → emits OrderCreated

PaymentService → listens, charges, emits PaymentCharged

InventoryService → listens, reserves stock, emits StockReserved

On failure: each service emits failure event → others compensate

Orchestration saga:

SagaOrchestrator drives each step, handles failures explicitly

Clearer failure handling, easier to debug

Option 3: Outbox Pattern

Write to DB + outbox table in same transaction

Outbox poller reads and publishes events to Kafka

Ensures at-least-once delivery without distributed transaction

Used with Saga or event-driven architectures

Option 4: CRDT / Eventual Consistency

Accept eventual consistency for non-critical data

Interview tip: "2PC is rarely used in modern microservices — too fragile. Saga pattern with the Outbox Pattern is the production standard. Mention both."

Caching & CDN

Q26. What are the caching strategies?

Cache
strategies

Medium

Cache-Aside (Lazy Loading):

App checks cache → miss → fetch from DB → populate cache → return

Pros: cache only what's needed, resilient to cache failure

Cons: cache miss = 3 trips (cache + DB + cache write), stale data

Read-Through:

Cache sits in front, fetches from DB on miss automatically

Pros: transparent to app

Cons: cold start (every key misses once)

Write-Through:

Write to cache + DB synchronously

Pros: always fresh cache

Cons: write latency (waits for both), cache pollution (write things never read)

Write-Behind (Write-Back):

Write to cache, async flush to DB

Pros: very fast writes

Cons: data loss risk (cache crash before flush)

Write-Around:

Write directly to DB, skip cache

Pros: avoids caching data written once and never read again

Cons: cold reads after write

Refresh-Ahead:

Proactively refresh cache before expiry based on predicted access

Pros: no cold misses for popular data

Cons: complex, may refresh data never read again

Production pattern (most common):

Read: Cache-Aside

Write: Write-Around or Write-Through with short TTL

Invalidation: delete on write (not update) + TTL as safety net

Interview tip: "The most asked question is cache invalidation. Best answer: 'Delete on write + TTL. Never update cached objects (causes race conditions). Short TTL as a safety net.'"

Q27. What is Redis and what are its data structures?

Redis
fundamentals

Easy

Redis: in-memory data structure store. Single-threaded (mostly).

Persistence: RDB (snapshot) or AOF (append-only log) or both.

Replication: primary-replica. Cluster: hash-slot-based sharding.

Data Structures:

String: SET/GET. Use: counters, cache any value, sessions

SET user:1:name "Alice"

INCR page:views:home

List: LPUSH/RPUSH/LPOP. Use: queues, recent activity

LPUSH recent:user:1 "item_id_42"

LRANGE recent:user:1 0 9 → last 10 items

Hash: HSET/HGET. Use: object fields, user profile

HSET user:1 name Alice age 30 email alice@example.com

HGETALL user:1

Set: SADD/SMEMBERS/SINTER. Use: unique visitors, tags, friends

SADD user:1:friends "2" "3" "4"

SINTER user:1:friends user:2:friends → mutual friends

Sorted Set: ZADD/ZRANGE/ZRANK. Use: leaderboards, rate limiting, timelines

ZADD leaderboard 1500 "alice" 1200 "bob"

ZREVRANGE leaderboard 0 9 → top 10

HyperLogLog: approximate unique count. Use: unique visitor count

PFADD visitors:today "user1" "user2"

PFCOUNT visitors:today → ~unique count, 0.81% error

Pub/Sub: publish-subscribe. Use: real-time notifications, chat

Streams: append-only log. Use: event sourcing, audit log

Lua scripts: atomic multi-command operations

Interview tip: "Redis Sorted Sets are the most interview-asked structure. Know: ZADD $O(\log N)$, ZRANK $O(\log N)$, ZRANGE $O(\log N + M)$. Used for leaderboards, rate limiters, delayed job queues."

Q28. What is cache eviction policy?Cache
management

Easy

When cache is full, eviction policy decides what to remove.

LRU (Least Recently Used): evict item not accessed for longest time
Best for: recently accessed data is likely to be re-accessed soon
Implementation: hash map + doubly-linked list ($O(1)$ operations)
Redis: allkeys-lru or volatile-lru

LFU (Least Frequently Used): evict item accessed fewest times
Best for: some items are accessed less but recently (new items)
LFU can evict a "new" popular item – needs decay mechanism
Redis: allkeys-lfu or volatile-lfu

FIFO: evict oldest item regardless of access
Simple but ignores access patterns

Random: randomly evict any item
Surprisingly competitive with LRU in some workloads

TTL-based: evict expired items (not really an eviction policy)
Always used in combination with above policies

Redis eviction policies:
noeviction: return error when full (bad for cache, ok for queue)
allkeys-lru: LRU across all keys (most common for pure cache)
volatile-lru: LRU among keys with TTL set
allkeys-random: random eviction
volatile-ttl: evict key with shortest TTL first

Rule: allkeys-lru is best for general caching.
volatile-lru when you have mixed use (cache + queue in same Redis).

Interview tip: "LRU is $O(1)$ with the right implementation (doubly linked list + hash map). A naive LRU that scans the cache is $O(N)$. Know the implementation from the LLD section."

Q29. What is a cache stampede and how do you prevent it?Cache
reliability

Hard

Cache stampede (thundering herd):
Popular cache key expires
100s of requests simultaneously miss cache
All hit DB simultaneously → DB overload → slow responses → more misses

Prevention strategies:

1. Mutex/Lock (single-flight):
First goroutine gets lock → fetches from DB → populates cache
Others wait for lock → get result from cache
Go: golang.org/x/sync/singleflight
Cost: adds latency for waiting requests
2. Probabilistic Early Expiration (PER):
Refresh cache slightly before TTL expires with some probability
 $P(\text{refresh})$ increases as expiry approaches
Key insight: no synchronization needed, just background refresh
3. Cache-warmup on deploy:
Pre-populate cache before TTL expires

Background job refreshes popular keys
Uses access logs to predict what to warm

4. Jittered TTL:

TTL = base_ttl + random(0, jitter_amount)
Prevents many keys expiring at the same time
Popular items: TTL=5min ± 30sec
Simple and effective

5. Stale-while-revalidate:

Serve stale data immediately
Trigger async background refresh
Client sees slightly old data but no stampede

Production: combine singleflight + jittered TTL + background refresh

Interview tip: "singleflight is the Go idiom. In other languages: Redis SETNX as a distributed lock. Jittered TTL is the simplest prevention — always add it."

Q30. What is a write-through vs write-around cache?

Write
strategies

Easy

Write-Through:

Write → cache AND DB simultaneously (synchronous)
Cache always consistent with DB
Every written item is cached (even if never read again)

Pros: cache hit on first read after write
Cons: write latency (waits for both), cache pollution
Use: read-heavy with frequent re-reads of written data

Write-Around:

Write → DB only (bypass cache)
Cache loaded lazily on first read (cache-aside pattern)

Pros: no cache pollution, no write latency penalty
Cons: read after write always misses cache (cold read)
Use: data written once and rarely read (logs, audit trails)

Write-Behind (Write-Back):

Write → cache only (async DB flush)

Pros: very fast writes, absorbs write bursts
Cons: data loss if cache fails before flush
Use: session data, shopping cart (acceptable to lose)
Mitigation: Redis AOF persistence + replica

Combined pattern:

Write: Write-Around (write direct to DB, delete/invalidate cache)
Read: Cache-Aside (check cache, miss → DB → populate cache)
Result: cache populated on demand, no stale data, no pollution

Interview tip: "The combined Write-Around + Cache-Aside is the safest production pattern. On write: delete cache key (not update). Cache is eventually populated when someone reads."

Q31. What is Redis Cluster and how does it shard data?

Redis scaling

Hard

Redis Cluster: horizontal scaling via hash slot sharding.

Total hash slots: 16,384

Key → CRC16(key) % 16384 → hash slot → node

3-node cluster: Node A: 0-5461, Node B: 5462-10922, Node C: 10923-16383

Hash tags: {user_id} → hash only user_id
 Forces related keys to same slot
 Example: {user:42}:profile and {user:42}:sessions
 → same slot → MULTI/EXEC transactions work

Scaling:

Add node: Redis rebalances slots automatically
 Remove node: drain slots first, then remove

Replication in cluster:

Each primary has 1+ replicas
 6 nodes = 3 primaries + 3 replicas (recommended minimum)

Limitations:

Multi-key operations: only if all keys in same slot (use hash tags)
 Lua scripts: all keys must be in same slot
 Pub/Sub: works across cluster
 No cross-slot transactions

When to use Redis Cluster vs single Redis:

Single Redis: up to ~100GB RAM, ~1M ops/sec (fast enough for most)
 Redis Cluster: >100GB data or need >1M ops/sec
 Many teams delay clustering too long – Redis Cluster adds complexity

Alternative: Redis Sentinel (HA without sharding)

Primary + replicas + Sentinel processes for automatic failover
 Simpler than Cluster, no sharding

Interview tip: "Know hash slots = 16,384. Know hash tags for co-location. Redis Cluster is overkill for most SDE2 designs — mention Sentinel for HA first, Cluster only for massive scale."

Q32. When do you cache at the application vs database vs CDN level?

Cache
placement

Medium

CDN cache (edge, global):

What: static assets, public API responses, HTML pages
 TTL: minutes to days
 Benefits: lowest latency (20ms vs 200ms), offload origin by 90%+
 Cache-Control: public, max-age=3600
 Invalidation: cache key = URL + version (bust with query param)

Application-level cache (in-process):

What: computed results, parsed configs, user session
 Where: memory inside the process (e.g., sync.Map in Go)
 TTL: seconds to minutes
 Benefits: 0ms (no network), perfect for hot lookup data
 Limits: not shared across instances, lost on restart

Distributed cache (Redis):

What: user sessions, rate limit counters, expensive DB query results
 Where: shared across all app servers
 TTL: seconds to hours
 Benefits: shared state, survives app restart, rich data structures
 Latency: ~1ms (loopback) to ~5ms (cross-AZ)

Database-level cache (query cache, buffer pool):

What: frequently accessed table pages, index pages
 Where: PostgreSQL shared_buffers (RAM)

TTL: managed by DB engine
 Benefits: automatic, no code changes
 Set shared_buffers = 25-40% of total RAM

Hierarchy: CDN → App cache → Redis → DB buffer pool → DB disk
 Each layer handles a subset of traffic, reducing pressure on the next.

Interview tip: "Design cache layers from outer to inner. CDN should absorb 90% of static traffic. Redis absorbs 80% of DB reads. DB only sees 10% of total requests. State these percentages."

Q33. How does cache invalidation work across multiple services?

Distributed
cache
invalidation

Hard

Problem: Service A writes to DB. Service B has cached the same data.
 How does Service B know to invalidate its cache?

Strategy 1: TTL only (simplest)

Every cache entry has a TTL (e.g., 5 minutes)
 Stale data serves until TTL expires
 Good enough for: user profiles, product catalog
 Not OK for: inventory, prices, balances

Strategy 2: Event-driven invalidation

Writer publishes "cache:invalidate:user:42" to message broker
 All services listening delete their local cache entry
 Challenge: at-least-once delivery → idempotent invalidation
 Implementation: Redis Pub/Sub or Kafka topic

Strategy 3: Write-Through invalidation

Any write to DB triggers cache delete in the same service
 Other services rely on TTL or subscribe to events

Strategy 4: Versioned cache keys

Key = "user:42:v7" where v7 is the current version
 On update: increment version in DB, new key on next read
 Old keys expire via TTL
 No explicit invalidation needed
 Trade-off: old versions hang around until TTL

Strategy 5: CDC (Change Data Capture) - Debezium

Debezium reads PostgreSQL WAL stream
 Publishes row changes to Kafka
 Cache invalidation service consumes Kafka → deletes Redis keys
 Fully decoupled, near-real-time

Interview tip: "The best production pattern: Delete-on-write (in same transaction as DB write) + short TTL (safety net) + event-driven invalidation for cross-service. Don't update cached values — always delete."

Q34. What is a Bloom filter and where is it used in caching?

Probabilistic
data
structures

Hard

Bloom filter: space-efficient probabilistic set membership test.
 "Definitely not in set" or "Probably in set" (false positives possible)
 No false negatives.

How it works:

- Bit array of size M, initialised to 0
- K hash functions

- Add element: set bits at k positions to 1
- Check: if all k positions are 1 → probably in set
- False positive rate $\approx (1 - e^{(-kn/m)})^k$

Properties:

- No deletions (use Counting Bloom Filter for deletions)
- False positive rate decreases with larger M
- $O(k)$ insert and lookup (k = number of hash functions)

Applications in caching:

- Cache negative results: "Does user_id 99999 exist?"
 - Without bloom filter: cache miss → DB query → "not found" → cache {99999: null}
 - With bloom filter: check bloom filter → "definitely not" → skip DB entirely
- Saves DB hit for non-existent keys

Real-world uses:

- Cassandra: bloom filter per SSTable to avoid reading wrong SSTable
- HBase: bloom filter on each HFile
- Redis: RedisBloom module
- Chrome: malicious URL detection (downloaded bloom filter)
- Akamai CDN: avoid caching one-time-access content

Space vs accuracy:

- 1% false positive rate: ~9.6 bits per element
- 0.1% false positive rate: ~14.4 bits per element
- 1M elements at 1% FPR: ~1.2 MB

Interview tip: "Bloom filters are asked in almost every big-tech HLD interview. Know: no false negatives, can have false positives, cannot delete (standard), $O(1)$ lookup. Use case: avoid DB lookup for non-existent keys."

Messaging & Queues

Q35. What is a message queue and when do you use one?

Async
messaging

Easy

Message queue: decouples producer from consumer asynchronously.

When to use:

- Decouple services (payment → email notification)
- Absorb traffic spikes (queue requests, process at steady rate)
- Async processing (video encoding, email sending)
- Fan-out (one event → multiple consumers)
- Retry logic (failed messages retried automatically)
- Rate limiting consumers (prevent overwhelming downstream)

Message delivery semantics:

- At-most-once: message may be lost, never duplicated (fire-and-forget)
- At-least-once: message may be duplicated, never lost (ack after processing)
- Exactly-once: no loss, no duplicates (hardest, most expensive)
- Idempotent consumers + at-least-once = effectively exactly-once

Pull vs Push:

- Pull (Kafka, SQS): consumer polls queue
 - Pros: consumer controls rate, no overwhelming
- Push (RabbitMQ, SNS): broker pushes to consumer
 - Pros: lower latency, simpler consumer
 - Cons: consumer can be overwhelmed

Ordering	Per partition	Per queue (roughly)
Throughput	1M+ msg/sec	100K msg/sec
Consumer Push?	No. Pull (long-poll)	Push (consumer-driven)
Routing	Partition key	Exchange + routing key
Use case	Stream processing, audit	Task queues, RPC, fanout

RabbitMQ strengths:

- Flexible routing (topic, fanout, direct, header exchanges)
- Request-reply (RPC) pattern
- Message priority queues
- Per-message TTL and dead letter exchange
- Better for task queues (Celery uses RabbitMQ)

Kafka strengths:

- Event sourcing (replay events)
- Stream processing (Kafka Streams, Flink)
- Multiple independent consumers of same events
- Extremely high throughput
- Long-term event log

When to choose:

"I need to send an email after payment" → RabbitMQ/SQS (simple task queue)

"I need multiple services to react to payment + replay + analytics" → Kafka

Interview tip: "RabbitMQ deletes messages after consumption. Kafka retains them. This is the fundamental difference — Kafka is a log, RabbitMQ is a queue. Replay = Kafka."

Q38. What is the outbox pattern?

Transactional
outbox

Hard

Problem: atomic write to DB + publish to Kafka — two separate systems.

If DB commit succeeds but Kafka publish fails → data inconsistency

```
// WRONG — not atomic
db.Exec("INSERT INTO orders ...") // may succeed
kafka.Publish("order.created", ...) // may fail → event lost
```

Outbox Pattern:

Write to DB + outbox table in ONE transaction

Separate process reads outbox → publishes to Kafka → marks as processed

```
// CORRECT
tx.Exec("INSERT INTO orders ...")
tx.Exec("INSERT INTO outbox (event_type, payload) VALUES (...)")
tx.Commit() // atomic — both or neither

// Outbox poller (separate goroutine/process)
rows := db.Query("SELECT * FROM outbox WHERE processed=false ORDER BY id")
for _, row := range rows {
    kafka.Publish(row.EventType, row.Payload)
    db.Exec("UPDATE outbox SET processed=true WHERE id=?", row.ID)
}
```

Implementations:

Polling: simple, slight delay

Debezium (CDC): reads PostgreSQL WAL → publishes to Kafka (near-real-time, no polling)

Transactional Outbox guarantees:

At-least-once delivery (idempotent consumers required)

No message loss

No phantom events (events only for committed transactions)

Interview tip: "The outbox pattern is the standard solution for 'write to DB and publish to Kafka atomically.' Every senior Go backend engineer should know this pattern cold."

Q39. What is backpressure in a messaging system?

Flow control

Medium

Backpressure: mechanism to prevent fast producers from overwhelming slow consumers.

Without backpressure:

Producer: 10,000 msg/sec
Consumer: 1,000 msg/sec
Queue grows 9,000 msg/sec → OOM → crash

With backpressure:

Consumer signals capacity to producer
Producer slows down or drops (fail-fast)

Strategies:

1. Bounded queue: queue has max size; producer blocks when full
Pro: simple
Con: producer blocks (cascades back)
2. Drop: discard messages when queue is full
Pro: producer never blocks
Con: data loss acceptable? (logs: yes; payments: no)
3. Rate limiting: limit producer to consumer's throughput
Pro: no loss, no blocking
Con: requires capacity estimation
4. Consumer groups + partition scaling (Kafka):
Add more consumer instances → more parallelism
Auto-scaling: CloudWatch alarm on consumer lag → add instances
5. Circuit breaker: stop producing if consumer is unhealthy

Kafka consumer lag:

Metric: consumer group lag = latest_offset - consumer_committed_offset
Alert threshold: lag > 10,000 → consumer can't keep up
Action: scale consumers (add instances or increase partitions)

Go implementation:

Bounded channel = backpressure
make(chan Job, 100) → producer blocks at 100
make(chan Job, 0) → unbuffered, perfect synchronisation

Interview tip: "Consumer lag is the key Kafka metric to monitor. Alert on lag > threshold → auto-scale consumers. Discuss this when asked about Kafka reliability."

Q40. What is event-driven architecture?

EDA

Medium

Event-driven architecture: services communicate via events (not direct calls).

Components:

Event producer: emits event when something happens
Event broker: Kafka, SNS, EventBridge
Event consumer: reacts to events asynchronously

Benefits:

Loose coupling: producer doesn't know about consumers
 Extensibility: add consumer without changing producer
 Resilience: consumer failure doesn't affect producer
 Audit trail: events are immutable facts (event log = audit log)

Patterns:

Event Notification: "order.placed" → email consumer reacts
 Event-Carried State Transfer: event contains full object data → no need to call producer's API
 Event Sourcing: events are the source of truth → derive state by replaying

Challenges:

Eventual consistency: consumers may lag
 Idempotency: events may be delivered twice → consumers must handle
 Ordering: Kafka guarantees order per partition only
 Schema evolution: changing event schema breaks consumers
 Use: Avro with Schema Registry or JSON with versioned fields

Commands vs Events:

Command: "CreateOrder" – imperative, one receiver, expects response
 Event: "OrderCreated" – past tense, broadcast, multiple receivers

Choreography vs Orchestration:

Choreography: services react to events independently (loose coupling)
 Orchestration: central coordinator directs services (easier to trace)

Interview tip: "Events should be in past tense ('OrderCreated', 'PaymentFailed'). Commands are imperative ('CreateOrder'). Getting the semantics right shows design maturity."

Q41. How do you handle duplicate messages (idempotency)?

Idempotency

Medium

At-least-once delivery means duplicates happen. Consumers must be idempotent.

Strategies:

1. Natural idempotency:

"Set user.email = X" → running twice has same result
 "Set order status to SHIPPED" → idempotent

2. Idempotency key (database):

Each message has unique ID (Kafka offset or UUID)
 Before processing: INSERT INTO processed_messages (msg_id) ON CONFLICT DO NOTHING
 If insert succeeds → process
 If insert fails (conflict) → already processed, skip

3. Conditional update:

UPDATE orders SET status='shipped'
 WHERE order_id=? AND status='processing'
 → Only updates if in expected state; noop if already shipped

4. Redis deduplication:

SET processed:{msg_id} 1 EX 86400 NX
 If SET succeeds → process
 If SET fails (key exists) → duplicate, skip

5. Event sourcing (naturally idempotent):

Each event has sequence number
 "Apply event 42 to aggregate" → if aggregate already at version 42+ → skip

Implementation in Go:

```
func (h *Handler) HandlePaymentProcessed(msg kafka.Message) error {
```

```

    key := fmt.Sprintf("processed:%s", msg.Key)
    if !redis.SetNX(key, 1, 24*time.Hour) {
        return nil // duplicate – skip
    }
    return h.processPayment(msg.Value)
}

```

Interview tip: "Idempotency is non-negotiable for payment processing. Always ask: 'What happens if this message is delivered twice?' If the answer is 'bad things,' you need idempotency."

Q42. What is a dead letter queue (DLQ)?

Error handling
in queues

Easy

Dead Letter Queue: separate queue for messages that fail processing.

When a message goes to DLQ:

- Exceeded max retry attempts
- Message could not be deserialized
- Consumer threw unretryable exception

Why DLQ is important:

Without DLQ: failed messages block queue (head-of-line blocking) or are dropped

With DLQ: bad messages isolated → main queue keeps flowing → alerts for investigation

DLQ workflow:

1. Consumer fails to process message
2. Message retried N times (exponential backoff)
3. After N failures → moved to DLQ
4. Alert triggered (PagerDuty/Slack)
5. Engineer investigates → fix bug
6. Replay messages from DLQ to main queue

Implementation (AWS SQS):

Main queue → maxReceiveCount=5 → DLQ

CloudWatch alarm: DLQ depth > 0 → alert

Implementation (Kafka):

No built-in DLQ

Consumer catches exception → manually publish to "orders.dlq" topic

Consumer group on DLQ topic for investigation/replay

Monitoring:

Alert on DLQ depth > 0 (immediate investigation)

DLQ depth trend (growing = systemic issue)

Retry policy:

Immediate retry: transient network error

Delayed retry (backoff): service temporarily unavailable

No retry (→ DLQ): bad message format, business rule violation

Interview tip: "Every queue in production needs a DLQ. Period. Not having one means failed messages are either dropped (data loss) or block the queue (availability issue)."

Q43. What is CQRS (Command Query Responsibility Segregation)?

CQRS

Hard

CQRS: separate read model from write model.

Traditional (CRUD):

Single model for reads and writes → compromises for both

CQRS:

Write side (Command): handles mutations, domain logic, writes to write DB
 Read side (Query): handles queries, optimised read model, can be different DB
 Sync: events propagate writes to read model asynchronously

Example – Order System:

Write model (PostgreSQL, normalised):

```
orders: (id, user_id, status, created_at)
order_items: (order_id, product_id, qty, price)
```

Read model (Elasticsearch or DynamoDB, denormalised):

```
order_summary: (order_id, user_name, item_names, total, status, created_at)
→ Optimised for dashboard query "show me my orders"
```

Sync: OrderCreated event → read model projector → upsert into Elasticsearch

Benefits:

- Read model optimised exactly for query pattern
- Independent scaling of reads and writes
- Different storage for different needs

Challenges:

- Eventual consistency: read model lags behind write model
- Complexity: two models to maintain
- More moving parts (event bus, projections)

With Event Sourcing (ES + CQRS):

- Write store = event log (append-only)
- Read store = projected views from events
- Any read model can be rebuilt by replaying events

Interview tip: "CQRS is overkill for simple CRUD. Recommend it when: read QPS >> write QPS by 100×, read query patterns are complex, or read and write need independent scaling."

Distributed Systems Concepts

Q44. What is consistent hashing?

Consistent
hashing

Hard

Problem: when you add/remove a server, naive hashing ($\text{server} = \text{hash}(\text{key}) \% N$) remaps almost ALL keys → massive cache miss storm.

Consistent Hashing:

- Hash servers and keys onto the same ring $[0, 2^{32})$
- Key assigned to NEAREST server clockwise on ring
- Adding server: only keys between (new server, predecessor) remapped
- Removing server: only keys on that server remapped to successor
- Impact: only K/N keys remapped ($K=\text{keys}$, $N=\text{servers}$)

Virtual nodes:

- Without VNodes: non-uniform key distribution (servers on ring unevenly)
- With VNodes: each server = 150 virtual nodes on ring
- Keys distributed evenly even with few real servers
- New server takes ~equal share from all existing servers

Used by:

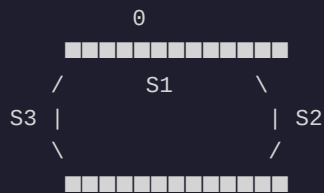
- Amazon DynamoDB (original Dynamo paper)
- Apache Cassandra (token ring)
- Redis Cluster (hash slots = discretized consistent hashing)

Ngix upstream hash
CDN server selection

Consistent hashing with bounded loads:

Extension: no server handles more than $(1+\epsilon) \times$ average load

Prevents hot servers when one server is over-assigned



Interview tip: "Draw the ring. Explain VNodes. Know the $O(\log N)$ lookup with sorted array of ring positions. Interviewers love to ask: 'What happens when a node fails?' → successor takes over."

Q45. What is a consensus algorithm and why is Raft important?

Consensus

Hard

Consensus: getting distributed nodes to agree on a value despite failures.

Foundation of: leader election, distributed locks, replicated state machines.

Paxos: original consensus algorithm. Notoriously hard to understand/implement.

Raft: designed for understandability. Same safety guarantees as Paxos.

Raft concepts:

States: Follower, Candidate, Leader

Term: monotonically increasing epoch (like a clock)

Log: ordered sequence of entries (commands to apply)

Leader election:

Follower → no heartbeat timeout → becomes Candidate

Candidate → sends RequestVote to all → majority vote → becomes Leader

Leader → sends heartbeats → prevents new elections

Log replication:

Leader → AppendEntries to all followers

Majority ACK → entry committed → applied to state machine

Quorum = $N/2 + 1$ (for $N=3$: 2 nodes must agree)

Safety: a committed entry will never be lost as long as majority survives

Used by:

etcd (Kubernetes control plane)

CockroachDB

TiKV

Consul

RabbitMQ (new quorum queues)

Quorum sizes:

3 nodes: tolerate 1 failure

5 nodes: tolerate 2 failures

Rule: tolerate F failures → need $2F+1$ nodes

Interview tip: "You don't need to implement Raft, but know: leader election, quorum writes, log replication. 'Why use etcd?' → distributed consensus for Kubernetes state. Raft is the foundation."

Q46. What is the two-phase commit (2PC) protocol?Distributed
transactions**Hard**

2PC: protocol for distributed atomic commit across multiple nodes.

Phase 1 (Prepare/Voting):

Coordinator → "Prepare?" → Participant 1, 2, 3

Each participant: lock resources, write to WAL, reply "Yes" or "No"

Phase 2 (Commit/Rollback):

All voted Yes → Coordinator → "Commit" → release locks

Any voted No → Coordinator → "Abort" → rollback

Problems with 2PC:

Blocking protocol: if coordinator crashes after Prepare but before Commit, participants hold locks indefinitely → deadlock

Single point of failure: coordinator is critical path

Latency: 2 round trips → 2 × network RTT overhead

Solution: coordinator writes Commit decision to WAL before sending
→ Recovers after restart

3PC (Three-Phase Commit): adds "PreCommit" phase to reduce blocking

Still has issues under network partitions → rarely used in practice

Practical alternatives:

Saga pattern: compensating transactions (no locking)

Outbox pattern: atomic DB write + event for eventual consistency

Google Spanner: TrueTime + 2PC with external consistency

When 2PC is appropriate:

Single-vendor, low-latency intra-datacenter (XA transactions in databases)

Not for cross-service microservices communication

Interview tip: "State the 2PC blocking problem immediately. 'Coordinator crashes in Phase 2 → participants blocked.' This shows you know the weakness. Then propose Saga as the modern alternative."

Q47. What is the difference between strong and eventual consistency?Consistency
models**Medium**

Consistency spectrum (strongest to weakest):

Linearizability (strongest):

Every operation appears atomic and instantaneous

All clients see same order of operations

As if only one copy of data

Cost: global coordination → high latency

Used by: Zookeeper, etcd, Google Spanner

Sequential Consistency:

All clients see same order of operations

But operations not necessarily instantaneous

Used by: some distributed memory systems

Causal Consistency:

Operations that are causally related appear in order

Concurrent operations may appear in different orders

Used by: MongoDB (causal sessions), DynamoDB (conditional writes)

Read-Your-Writes:

- Client always sees its own writes
- Other clients may see stale data
- Practical minimum for user-facing systems

Eventual Consistency (weakest):

- If no new writes, all replicas will eventually converge
- No guarantee on when convergence happens
- Used by: DynamoDB, Cassandra, DNS

Choosing consistency:

- Inventory, balances: Linearizable (last-item race condition)
- Social feed: Eventual (user sees your post 200ms later? Fine)
- Shopping cart: Read-Your-Writes (you add item, you see it)
- Friend list: Causal (if A unfriends B, B shouldn't see A's posts)

Interview tip: "Eventual consistency doesn't mean 'broken.' DNS is eventually consistent and works fine. The question is: 'What's the business impact of stale data?' Frame your choice around that."

Q48. What is a distributed lock and when do you need one?Distributed
locking**Hard**

Distributed lock: ensure only one process executes a critical section across multiple machines.

When needed:

- Scheduled job must run on exactly one node
- Inventory: decrement only once despite concurrent requests
- Idempotent processing: exactly-once execution
- Leader election

Redis-based lock (Redlock / SET NX):

- SET lock:{resource} {token} NX PX {ttl_ms}
- NX = only set if not exists
- PX = expire in milliseconds
- token = random UUID (identify owner)
- Release: Lua script to delete only if token matches

Problems with Redis single-node lock:

- If Redis primary fails before replica syncs → two processes get lock
- Solution: Redlock (acquire lock on $N/2+1$ Redis nodes)

ZooKeeper-based lock (stronger):

- Create ephemeral sequential node
- Watch predecessor node
- On predecessor deletion → acquire lock
- Fault-tolerant: ephemeral node deleted if holder dies (session expiry)

Lock with lease renewal:

- TTL = 30s, renew every 10s
- Background goroutine sends heartbeat
- If process dies, lock expires automatically

Fencing token:

- Each lock acquisition has increasing ID
- Storage system rejects requests with old fencing token
- Prevents split-brain (two processes think they hold lock)

Interview tip: "Single Redis node lock has failure modes. Mention fencing tokens as the production-safe solution. 'Even if two processes think they have the lock, the fencing token ensures only one actually succeeds at the storage'"

`layer."`

Q49. What is a service mesh?

Service mesh

Hard

Service mesh: infrastructure layer for service-to-service communication.

Handles: load balancing, service discovery, mTLS, retries, circuit breaking, observability.

Architecture: sidecar proxy (Envoy) injected into every pod.

Application → Sidecar → Network → Sidecar → Application

Application doesn't know about network concerns.

Features:

Service discovery: sidecar knows about all services via control plane

Load balancing: L7 (request-level) with health awareness

mTLS: automatic mutual TLS between all services

Retries: configurable retry policy per route

Circuit breaking: auto-open circuit on error threshold

Traffic splitting: canary deployments (5% to new version)

Observability: metrics, traces, logs – automatic, without code changes

Rate limiting: per-service, per-client rate limits

Tools:

Istio (most popular): complex but full-featured

Linkerd: lightweight, Go-based, simpler than Istio

Envoy: the sidecar proxy used by both

When to use a service mesh:

>10 microservices

Compliance requires mTLS everywhere

Need canary deployments

Cross-language services (mesh is language-agnostic)

When NOT to use:

Small number of services (overkill)

Sidecar adds ~2-5ms latency + memory overhead

Team unfamiliar with service mesh complexity

Interview tip: "Don't bring up service mesh unless asked or for very large systems (50+ services). For an SDE2 interview system design, say 'we'd add Istio as the system grows' rather than designing it upfront."

Q50. What is service discovery?

Service
discovery

Medium

Service discovery: how services find each other's network addresses.

Challenge: in dynamic systems (Kubernetes), service IPs change constantly.

Client-side discovery:

Client queries service registry → gets list of instances → load balances

Examples: Netflix Eureka + Ribbon (legacy)

Pros: client controls LB algorithm

Cons: registry logic in every client

Server-side discovery:

Client → Load Balancer/API Gateway → registry → routes to instance

Examples: AWS ALB, Kubernetes kube-proxy, Nginx

Pros: client is simple

Cons: extra hop

Service Registry:

Q52. What is rate limiting at the API gateway level?

Rate limiting

Medium

Rate limiting: control request rate to protect services from abuse.

Where to implement:

API Gateway (Kong, AWS API Gateway, Nginx): before requests reach services

Service level: per-service limits (defense in depth)

Application level: per-feature limits

Algorithms (see LLD section for code):

Token bucket: bursty allowed, average rate enforced

Sliding window log: accurate, high memory (timestamp per request)

Sliding window counter: approximation, low memory

Dimensions:

Per IP: abuse/DDoS protection

Per API key: fair use for external APIs

Per user: prevent single user from hogging resources

Per endpoint: expensive endpoints have lower limits

Global: protect downstream databases

Response:

HTTP 429 Too Many Requests

Retry-After header: seconds until reset

X-RateLimit-Limit: allowed per window

X-RateLimit-Remaining: left in current window

X-RateLimit-Reset: Unix timestamp of window reset

Distributed rate limiting (multiple gateway nodes):

Centralized counter: Redis INCR (atomic, consistent)

Local counter: per-node (inaccurate under load balancing)

Approximate: token bucket with Redis, tolerate slight over-counting

Tiered limits (API product):

Free tier: 100 req/min

Pro tier: 10,000 req/min

Enterprise: custom

Interview tip: "Always mention distributed rate limiting requires Redis. Explain the trade-off: Redis-based is slightly slower (+1ms) but correct across all nodes. Local-only can be 10× inaccurate with round-robin LB."

Q53. What is a circuit breaker pattern?

Circuit breaker

Medium

Circuit breaker: prevent cascade failures by failing fast.

States:

Closed (normal): requests pass through, failures counted

Threshold: 5 failures in 60s → Open

Open (failing): all requests fail immediately (no service call)

After timeout (30s) → Half-Open

Half-Open (probing): one test request allowed

Success → Closed; Failure → Open again

Benefits:

Fast failure: 1ms rejection vs 30s timeout

Service recovery: allows downstream to recover (no constant hammering)

Cascade prevention: service A failing doesn't take down service B

Metrics to track:

Error rate: failures / total requests in window
 Latency p99: slow responses count as failures
 Success in half-open: needed to return to Closed

Libraries:

Go: github.com/sony/gobreaker, github.com/afex/hystrix-go
 Java: Resilience4j, Hystrix (deprecated)
 .NET: Polly

Configuration:

ConsecutiveFailures: 5 (open after 5 consecutive failures)
 Interval: 60s (rolling window)
 Timeout: 30s (time in Open before trying Half-Open)
 MaxRequests: 1 (requests allowed in Half-Open)

Combined with retry:

Retry: transient errors (connection reset, 503) – max 3 times
 Circuit breaker: sustained errors → stop retrying entirely
 Rule: circuit breaker OUTSIDE retry loop

Interview tip: "State: 'Circuit breaker prevents retry storms. Without it, retrying a broken service creates a DDoS against the recovery.' This shows systems thinking beyond the pattern itself."

Q54. What is a distributed tracing system?

Observability

Medium

Distributed tracing: track a request's journey across multiple services.

Concepts:

Trace: full journey of a request (root span + all child spans)
 Span: single operation with timing (DB query, HTTP call, function)
 Trace ID: unique ID propagated across all services
 Span ID: unique ID for each operation
 Parent span ID: links child span to parent

Context propagation (HTTP headers):

X-Trace-ID: 7f3a9b...
 X-Span-ID: a3c1f8...
 X-Parent-Span-ID: 9b2e4d...
 W3C TraceContext standard: traceparent header

Flow:

```
Client → Service A (creates trace, span 1)
      Service A → DB (span 2, parent=span 1)
      Service A → Service B (span 3, parent=span 1)
                  Service B → Redis (span 4, parent=span 3)
```

OpenTelemetry (OTel): vendor-neutral instrumentation standard

SDK: instrument your code once
 Exporter: send to Jaeger, Zipkin, Honeycomb, Datadog, etc.

Go instrumentation:

otelhttp: auto-instrument HTTP servers and clients
 otelgrpc: auto-instrument gRPC
 otelsql: auto-instrument database/sql

Benefits:

Find slow service in request chain
 Debug intermittent errors
 Understand service dependencies
 Measure p99 latency per service

Tools: Jaeger (open source), Zipkin (open source), Honeycomb, Datadog APM

Interview tip: "Mention OpenTelemetry as the standard — instrument once, send anywhere. The ability to trace a 'slow checkout' across 10 services is what makes distributed tracing invaluable."

Q55. What is the Saga pattern?

Distributed
transactions

Hard

Saga: sequence of local transactions with compensating transactions for failures.
Alternative to 2PC for distributed transactions across microservices.

Example: Order placement

1. Order Service: create order (status=pending)
2. Payment Service: charge card
3. Inventory Service: reserve stock
4. Shipping Service: create shipment

If step 3 fails:

- Compensate step 2: refund payment
- Compensate step 1: cancel order

Choreography (event-driven):

OrderCreated → PaymentService charges → PaymentCharged
PaymentCharged → InventoryService reserves → StockReserved
StockReserved → ShippingService creates shipment → ShipmentCreated

Failure: StockOutOfStock event →

- PaymentService listens → RefundPayment
- OrderService listens → CancelOrder

Orchestration (central coordinator):

SagaOrchestrator → calls each service in sequence
Tracks state of entire saga
On failure → calls compensating transactions
Easier to understand, debug, monitor

Challenges:

- Compensating transactions must be idempotent
- Business logic for compensation can be complex
- Partial failures: need clear rollback semantics
- No isolation: interim states visible (order=pending-payment)

Tools:

- Temporal: workflow engine for sagas (Go SDK available)
- AWS Step Functions: managed saga orchestration
- Conductor (Netflix): open source orchestration

Interview tip: "Choreography = loose coupling, harder to trace. Orchestration = central control, easier to debug. Recommend orchestration via Temporal for complex sagas. Mention it by name."

Q56. What is leader election and how does it work?

Leader
election

Hard

Leader election: choose one node from a group to be the primary.
Used for: scheduled job execution, single-writer constraint, coordinator role.

ZooKeeper (Curator recipe):

- All nodes try to create /election/node_ (ephemeral sequential)
- Node with smallest sequence number = leader
- Others watch the node just before them

When leader dies, ephemeral node deleted → next watcher becomes leader

etcd (used in Kubernetes):

Leader acquires lease on a key: PUT /leader/{service} {nodeID} lease=TTL
 Other nodes watch /leader/{service}
 Lease expiry → race to acquire → one wins
 Kubernetes uses this for controller-manager leader election

Redis (Redlock):

SETNX leader:{service} {nodeID} PX {ttl}
 Renew every TTL/3
 On expiry → next node acquires

Kubernetes leader election (Go):

client-go/tools/leaderelection
 Uses configmap or lease resource as lock
 Callback: OnStartedLeading / OnStoppedLeading

Failure scenarios:

Leader dies: lease expires → new leader elected
 Network partition: two leaders possible (split-brain)
 Solution: fencing tokens + quorum-based lock
 False leader: leader thinks it's leader but network isolated
 Solution: write to quorum, not just leader

Time: leader election takes 1-2× lease duration (30-60s typical)

Interview tip: "Leader election is how Kubernetes controller-manager works. If asked 'how do you run a cron job exactly once in a cluster?' — leader election. etcd or a DB-based lock are production answers."

Microservices & APIs

Q57. What is REST vs gRPC?

API protocols

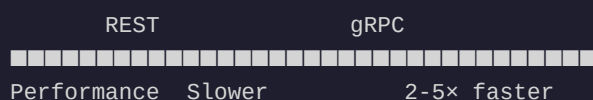
Easy

REST (Representational State Transfer):

Protocol: HTTP/1.1 or HTTP/2
 Format: JSON (human-readable, larger)
 Style: resource-based URLs, HTTP verbs (GET/POST/PUT/DELETE)
 Type safety: none (JSON is weakly typed)
 Streaming: limited (HTTP chunked or Server-Sent Events)
 Browser support: native
 Tooling: curl, Postman, OpenAPI/Swagger
 Use: public APIs, browser clients, mobile apps

gRPC (Google Remote Procedure Call):

Protocol: HTTP/2 (multiplexed, header compression)
 Format: Protocol Buffers (binary, smaller, faster)
 Style: service + method definitions (strongly typed)
 Type safety: enforced by protobuf schema
 Streaming: native (unary, server, client, bidirectional)
 Browser support: requires grpc-web proxy
 Tooling: grpcurl, generated clients for all languages
 Use: internal microservices, high-performance, streaming



Type safety	None	Strong (protobuf)
Streaming	Limited	Native 4 modes
Browser	Native	Needs proxy
Schema	Optional (OAS)	Required (.proto)
Readability	Human-readable	Binary (not debuggable)
Code gen	Optional	Standard (protoc)

Typical architecture:

External API (mobile/web): REST (JSON) — human-readable, firewall-friendly

Internal services: gRPC — faster, type-safe, streaming

API Gateway: translates REST → gRPC (transcoding)

Interview tip: "Use gRPC for internal microservice communication — it's 2-5× faster and type-safe. Use REST for external APIs. This is the standard production architecture."

Q58. What is GraphQL and when do you use it?

API design

Medium

GraphQL: query language for APIs. Client specifies exactly what data it needs.

Advantages:

No over-fetching: get only requested fields

No under-fetching: single request for nested data (vs multiple REST calls)

Strongly typed schema (SDL — Schema Definition Language)

Single endpoint: POST /graphql

Introspection: clients can query schema itself

Real-time: subscriptions (WebSocket)

REST vs GraphQL example:

REST to get post with author and comments:

GET /posts/42 → {post data}

GET /users/7 → {author data}

GET /posts/42/comments → {comments}

= 3 requests, over-fetching (unused fields)

GraphQL:

```
query {
  post(id: 42) {
    title content
    author { name avatar }
    comments(first: 5) { text createdAt }
  }
}
```

= 1 request, exactly what's needed

Challenges:

N+1 problem: nested resolvers → use DataLoader

Complex caching: responses vary by query structure

Authorization: per-field auth complex

File upload: not native in spec

Schema evolution: additive only (can't remove fields)

When to use:

Mobile apps: reduce bandwidth (battery + data)

Complex/nested data (social networks)

Rapid frontend iteration (no backend changes for new fields)

API serving multiple clients with different needs

When NOT to use:

Simple CRUD API

File uploads/downloads

Streaming (use gRPC or WebSocket)

Interview tip: "GraphQL's killer feature is the client specifying exactly what it needs. DataLoader is mandatory for any nested GraphQL resolver — otherwise N+1 queries destroy performance."

Q59. What is an API Gateway pattern?

API Gateway

Medium

API Gateway: single entry point for all clients.

Handles: routing, auth, rate limiting, SSL, transformation, caching.

Responsibilities:

Routing: /users → UserService, /orders → OrderService

Authentication: verify JWT/API key before forwarding

Rate limiting: per IP/user/API key

SSL termination: HTTPS → HTTP internally

Request/response transformation: REST → gRPC, JSON → Protobuf

Load balancing: distribute across service instances

Caching: cache GET responses

Monitoring: centralized metrics and logging

Circuit breaking: fail fast if downstream unhealthy

BFF (Backend for Frontend):

Separate API Gateway per client type

Mobile BFF: smaller payloads, fewer fields

Web BFF: richer data, more features

Avoids compromise in a single gateway

Tools:

Open source: Kong, Traefik, Nginx

Cloud managed: AWS API Gateway, Azure APIM, GCP Apigee

Service mesh: Istio (handles service-to-service)

Anti-patterns:

Too much business logic in gateway (it should route, not compute)

Single gateway as SPOF (run multiple instances behind LB)

Synchronous gateway for every microservice call (consider async for non-critical)

Design:

Client → [API Gateway: auth + rate limit + route]

→ [UserService] [OrderService] [PaymentService]

Interview tip: "API Gateway is a must in any microservices design. It prevents N×M connections (N clients × M services). Without it: every service must implement auth, rate limiting, SSL — code duplication."

Q60. What are the microservices decomposition patterns?

Microservices
design

Medium

Decompose by business capability:

Domain-aligned services: UserService, OrderService, PaymentService

Each service owns its data (bounded context)

Communication: events or APIs

Best for: stable, well-understood domain

Decompose by subdomain (DDD):

Core domain: competitive advantage (pricing engine, recommendation)

Supporting: generic but important (auth, notifications)

Generic: commodity (email, payments) → use third-party

Strangler Fig pattern (migration):

Legacy monolith → incrementally replace with microservices

New service deployed alongside → traffic gradually shifted
Old monolith functionality removed piece by piece
No big-bang rewrite → risk-free migration

Service granularity guidelines:

Too fine-grained:

- Network overhead for every interaction
- More deployment complexity
- Distributed transactions required

Too coarse-grained:

- Low cohesion, mixed concerns
- Single service can't be scaled independently

Right size: team can own it, deploys independently, single business function

Database patterns:

- Database per service (ideal): complete isolation
- Shared database (anti-pattern): tight coupling
- Shared schema (compromise): different tables, same DB

Inter-service communication:

- Sync: gRPC (low latency, immediate response needed)
- Async: Kafka (decouple, resilience, fan-out)

Interview tip: "The two-pizza rule: team that owns a service should be feedable with two pizzas. If the service needs 20 engineers, it's too big. If it needs 0.1 engineers, it's too small."

Q61. What is the difference between API versioning strategies?

API versioning

Easy

URL path versioning: /api/v1/users, /api/v2/users

- Pros: explicit, easy to route, easy to test
- Cons: URL changes, clients must update

Most common approach

Query parameter: /users?version=2

- Pros: URL stable, version optional
- Cons: less explicit, can be forgotten

Header versioning: Accept: application/vnd.myapi.v2+json

- Pros: URL stable, RESTful
- Cons: less visible, harder to test with browser

Content negotiation: Accept: application/vnd.company.resource-v2+json

- Pros: full REST compliance
- Cons: verbose, complex

Semantic versioning for APIs:

- Major (v1 → v2): breaking changes (remove fields, change types)
- Minor (v1.1 → v1.2): additive changes (new optional fields)
- Patch: bug fixes, no contract change

Breaking vs non-breaking changes:

- Breaking: remove field, change field type, rename endpoint
- Non-breaking: add optional field, add new endpoint, add optional header

API deprecation:

- Sunset header: Sunset: Wed, 11 Nov 2025 23:59:59 GMT
- Deprecation-Notice: deprecated API, migrate to v2
- Minimum deprecation period: 6-12 months for external APIs

Interview tip: "URL path versioning (/v1/, /v2/) is the de-facto standard. GraphQL avoids versioning by making all changes additive. Mention that your API Gateway can route v1 → old service, v2 → new service."

Q62. What is idempotency in APIs?

Idempotency

Medium

Idempotent operation: calling it multiple times = same result as calling once.

HTTP methods:

GET: naturally idempotent (just a read)
PUT: replace resource – idempotent
DELETE: delete resource – idempotent (second call: 404 or 200, same state)
POST: NOT idempotent (creates new resource each time)
PATCH: depends on implementation

Making POST idempotent (Idempotency-Key):

Client generates UUID per operation
Sends: POST /payments + Idempotency-Key: uuid-123
Server: check if uuid-123 already processed
First time: process payment, store {uuid-123: response}
Subsequent: return stored response immediately

Database implementation:

```
CREATE TABLE idempotency_keys (  
  key VARCHAR(255) PRIMARY KEY,  
  response JSON,  
  expires_at TIMESTAMP  
);  
  
INSERT INTO idempotency_keys (key, response, expires_at)  
VALUES (?, ?, NOW() + INTERVAL '24 hours')  
ON CONFLICT (key) DO NOTHING  
RETURNING key;
```

If RETURNING returns row → first call → process
If RETURNING returns nothing → duplicate → return stored response

Stripe uses Idempotency-Key header for all payment APIs.

Used by: Stripe, Twilio, Braintree.

Interview tip: "Idempotency-Key is mandatory for payment APIs. Network timeouts mean the client retries — without idempotency, you charge the card twice. Stripe's implementation is the gold standard."

Q63. What is the strangler fig pattern?

Migration

Medium

Strangler Fig: incrementally replace a legacy system without big-bang rewrite.
Named after the fig tree that grows around and replaces a host tree.

Steps:

1. Deploy new service alongside monolith
2. Route SOME traffic to new service (start with non-critical features)
3. Gradually increase traffic to new service
4. Remove corresponding code from monolith
5. Repeat until monolith is gone

Implementation:

API Gateway / Nginx as router:
/api/users/profile → new UserService (v2)
/api/users/legacy → old monolith (v1)

Feature flags:


```
if featureFlag("new_user_service") { callNewService() }
else { callMonolith() }
```

Strangler Facade: proxy in front of both
Routes based on feature, URL, or user segment

Traffic migration:

Start: 1% to new service (canary)
Monitor: errors, latency, correctness
Increase: 10% → 50% → 100%
Rollback: flip feature flag if issues

Database migration:

Dual-write: write to both old and new DB
Verify: compare outputs
Cut-over: read from new DB
Stop writing to old DB

Challenges:

Two systems running in parallel (operational overhead)
Data consistency during transition
Performance regression can be hard to spot

Interview tip: "For any 'migrate from monolith' question, the answer is Strangler Fig. Never big-bang rewrite — too risky. Gradual migration lets you validate at each step."

Q64. What is a health check endpoint and why is it important?

Operational

Easy

Health check: endpoint indicating whether a service is healthy.

Types:

Liveness probe (/live):
Is the process alive? (not deadlocked)
Returns 200: process is alive
Returns 500: process should be restarted
Simple: if we can respond, we're alive

Readiness probe (/ready):
Can we serve traffic? (DB connected, cache warmed)
Returns 200: ready for traffic
Returns 503: not ready — remove from LB rotation
Checks: DB connection, Redis connection, required dependencies

Startup probe:
Has initial startup completed?
Delays liveness/readiness until startup done
Prevents killing slow-starting services

Implementation in Go:

```
func (s *Server) livenessHandler(w http.ResponseWriter, r *http.Request) {
    w.WriteHeader(http.StatusOK)
    json.NewEncoder(w).Encode(map[string]string{"status": "ok"})
}

func (s *Server) readinessHandler(w http.ResponseWriter, r *http.Request) {
    ctx, cancel := context.WithTimeout(r.Context(), 2*time.Second)
    defer cancel()
    if err := s.db.PingContext(ctx); err != nil {
        w.WriteHeader(http.StatusServiceUnavailable)
        json.NewEncoder(w).Encode(map[string]string{"status": "db_unavailable"})
    }
}
```

```

        return
    }
    w.WriteHeader(http.StatusOK)
}

```

Kubernetes:

livenessProbe: restarts container if failing
 readinessProbe: removes pod from Service endpoints if failing
 Both check /live and /ready respectively

Interview tip: "Liveness vs readiness is a common Kubernetes interview question. Liveness = 'should this pod be restarted?' Readiness = 'should this pod receive traffic?' Don't conflate them."

Q65. What is canary deployment?

Deployment
strategy

Medium

Canary deployment: deploy new version to small subset of users first.
 Named after canary-in-coal-mine safety test.

Strategies:

Percentage-based: 1% → 5% → 25% → 100%
 User segment: internal users → beta users → all users
 Geographic: deploy to us-west-2 first, then other regions
 Random: $\text{hash}(\text{user_id}) \% 100 < 5 \rightarrow \text{new version}$

Implementation:

Kubernetes + Argo Rollouts:

```

progressDeadlineSeconds: 600
steps:
- setWeight: 5      # 5% to canary
- pause: {duration: 10m}
- setWeight: 20     # 20% to canary
- pause: {duration: 10m}
- setWeight: 100    # full rollout

```

Nginx/Envoy traffic splitting:

```

upstream old { server old-service:8080 weight=95; }
upstream new { server new-service:8080 weight=5; }

```

Feature flags (LaunchDarkly, Unleash):

```

if featureFlag.Enabled("new-checkout", userID) { → new }
else { → old }

```

Monitoring during canary:

Error rate: canary error rate must be $\leq$ production
 Latency: p99 canary $\leq$ production p99
 Business metrics: conversion rate, revenue per session
 Automatic rollback if thresholds breached

Blue-Green deployment (alternative):

Two identical environments: Blue (current) and Green (new)
 Switch LB from Blue to Green instantly
 Instant rollback: switch back to Blue
 Cost: 2× infrastructure

Interview tip: "Canary deployment is how Netflix/Uber/Amazon ship code safely. Combined with automatic rollback on error rate increase, it catches bugs before they affect all users."

Observability & Reliability

Q66. What are the three pillars of observability?

Observability

Easy

Observability: ability to understand internal state from external outputs.

Three pillars:

1. Metrics (quantitative, aggregated):

What: numeric measurements over time

Examples: QPS, error_rate, p99_latency, memory_usage, cache_hit_rate

Storage: Prometheus (pull-based), StatsD (push-based), CloudWatch

Visualization: Grafana dashboards

Alerting: alert when error_rate > 1% for 5min

2. Logs (events, text):

What: timestamped records of events

Examples: "user 42 logged in", "payment failed: insufficient funds"

Structured logging: JSON (queryable by field)

Storage: Elasticsearch, Loki, CloudWatch Logs, Splunk

Best practice: correlation ID / trace ID in every log line

3. Traces (distributed, causal):

What: end-to-end request journey across services

Examples: checkout → payment (50ms) → inventory (10ms) → shipping (20ms)

Storage: Jaeger, Zipkin, Honeycomb, Datadog APM

Standard: OpenTelemetry (instrument once, export anywhere)

The three together:

Metrics tell you SOMETHING is wrong (error rate up)

Logs tell you WHAT went wrong (specific error messages)

Traces tell you WHERE it went wrong (which service/span)

RED method (services):

Rate: requests per second

Errors: error rate

Duration: latency distribution

USE method (resources: CPU, disk, network):

Utilization: % resource used

Saturation: queue depth, waiting

Errors: error rate

Interview tip: "Always mention all three pillars in any reliability discussion. Add: 'We'd use OpenTelemetry for unified instrumentation — instrument once, send to any backend.'"

Q67. What is SLI, SLO, and SLA?

SRE concepts

Easy

SLI (Service Level Indicator): measurable metric of service behavior

Examples: availability, latency, error rate, throughput

SLO (Service Level Objective): target value for SLI

Examples: "99.9% availability", "p99 latency < 200ms", "error rate < 0.1%"

Internal goal – NOT a customer commitment (that's SLA)

SLA (Service Level Agreement): contractual commitment with consequences

If SLA breached: financial penalties, service credits

Example: "99.9% availability or 10% service credit"

$SLA \leq SLO$ (SLO is stricter – gives buffer before SLA breach)

Error budget:

Error budget = 1 - SLO
 99.9% SLO → 0.1% error budget = 8.77 hours/year
 Error budget consumed: actual downtime / total budget
 If error budget consumed → freeze new deployments
 If error budget healthy → ship faster, take more risk

Practical SLOs:

Availability: 99.9% (3 nines) for most services
 Latency p99: 200ms for APIs, 1s for complex queries
 Error rate: < 0.5%
 Freshness: data updated within 5 minutes

Multi-window alerting:

Alert fast: 2% error rate in last 5 minutes (fast burn)
 Alert slow: 0.5% error rate in last 6 hours (slow burn)
 Both needed: fast catches spikes, slow catches slow leaks

Interview tip: "Error budget is the key concept interviewers test. 'How do you balance feature velocity with reliability?' → Error budget: when it's healthy, ship features; when it's exhausted, fix reliability."

Q68. What is the difference between horizontal and vertical partitioning?

Data
partitioning

Medium

Horizontal partitioning (Sharding):

Split rows across multiple tables/databases
 Each shard has same schema, different data
 Example: users 1-1M in shard1, 1M-2M in shard2
 Reduces: rows per table, query time, storage per node
 Increases: write throughput, can store more data

Vertical partitioning:

Split columns across multiple tables
 Related/frequently-accessed columns together
 Example:
 users_core: (id, name, email, created_at) – queried frequently
 users_profile: (id, bio, avatar, settings) – queried less often
 users_metrics: (id, login_count, last_ip) – updated frequently
 Reduces: table width, I/O for narrow queries
 Useful for: wide tables, column access patterns differ

Functional partitioning:

Split by feature/domain into separate databases
 user_db: all user data
 order_db: all order data
 payment_db: all payment data
 = microservices DB-per-service pattern

Partition pruning (query optimisation):

PostgreSQL table partitioning:
 PARTITION BY RANGE (created_at)
 Partition for each month
 Query for last month only scans last month's partition
 Huge savings for time-series data

Partition key selection:

Access pattern drives key selection
 If 90% of queries filter by user_id → shard/partition by user_id

Interview tip: "Horizontal = row-based sharding. Vertical = column splitting. Know both. PostgreSQL declarative partitioning (RANGE/LIST/HASH) is the production implementation of horizontal partitioning."

Q69. What is chaos engineering?

Reliability
engineering

Medium

Chaos engineering: intentionally inject failures to find weaknesses before they cause outages.

Principles (Netflix Chaos Monkey paper):

- Build hypothesis: "System maintains 99.9% availability when one AZ fails"
- Vary real-world events: server crash, network partition, high latency
- Run in production (or staging): real behavior, real dependencies
- Minimize blast radius: start small, monitor closely
- Automate to run continuously

Types of failure injection:

- Resource failure: kill random pods/VMs (Chaos Monkey)
- Network: add latency, packet loss, partition between services
- CPU/Memory: spike CPU, fill memory
- Clock skew: advance system clock
- Dependency failure: make Redis unresponsive, DB connection refused
- AZ failure: shut down all instances in one availability zone

Tools:

- Chaos Monkey (Netflix): kills EC2 instances randomly
- Gremlin: comprehensive chaos platform
- Chaos Toolkit: open source, multiple plugins
- Litmus (Kubernetes): CNCF chaos engineering tool
- Istio fault injection: inject HTTP errors/delays at mesh level

Controlled experiment:

- Define: "payment service handles Redis failure gracefully"
- Baseline: measure normal p99 latency, error rate
- Inject: make Redis unresponsive for 60 seconds
- Observe: does circuit breaker open? does it fall back to DB?
- Conclusion: pass/fail against hypothesis

Game day: scheduled exercise where team intentionally causes failures

Interview tip: "Chaos engineering is practiced by Netflix, AWS, Google. For SDE2: know what it is and why. 'We use Chaos Monkey to ensure our circuit breakers and fallbacks actually work in production.'"

Q70. What is on-call and incident management?

Operational
excellence

Easy

On-call: engineers are reachable 24/7 to respond to production incidents.

Severity levels:

- SEV1 (Critical): complete outage, data loss, security breach
 - Response: immediate, all hands
 - Notify: CTO, customers, status page
- SEV2 (Major): major feature broken, significant user impact
 - Response: < 15 minutes
 - Notify: engineering manager
- SEV3 (Minor): degraded performance, partial feature unavailable
 - Response: business hours
- SEV4 (Cosmetic): UI issue, minor bug
 - Schedule: next sprint

Incident response steps:

1. Acknowledge: claim the incident, prevent duplicate response
2. Assess: understand impact (how many users? revenue?)
3. Communicate: update status page, notify stakeholders
4. Investigate: use metrics + traces + logs to find root cause
5. Mitigate: fastest fix (rollback, feature flag off, scale up)
6. Resolve: permanent fix
7. Post-mortem: blameless analysis within 48 hours

Runbook: documented step-by-step response procedure for known incidents

"Redis is full → check largest keys → flush expired → page DBA"

Post-mortem (blameless):

Timeline of events

Root cause analysis (5 Whys)

Contributing factors

Action items (prevent recurrence)

No blame, no finger-pointing – systems, not people

Tools: PagerDuty, OpsGenie (alerting), Statuspage (communication)

Interview tip: "Blameless post-mortem culture prevents cover-up and enables learning. 'We don't fire people for outages — we fix the system.' This is the SRE philosophy."

Q71. What is blue-green deployment?

Deployment

Easy

Blue-Green: two identical production environments.

Blue: currently live (100% traffic)

Green: new version deployed (0% traffic)

Process:

1. Deploy new version to Green (while Blue serves all traffic)
2. Run smoke tests against Green
3. Flip LB: route 100% traffic to Green
4. Blue kept warm for rollback
5. After validation: decommission Blue / make it next staging

Rollback: flip traffic back to Blue (seconds, not hours)

vs Canary:

Blue-Green: instant switch, all-or-nothing

Canary: gradual traffic shift, catches issues earlier

Challenges:

Database migrations: schema changes must be backward compatible

Blue and Green may run simultaneously during switch

Solution: expand-contract migrations (multi-step schema changes)

2× infrastructure cost during deployment

Session state: active sessions on Blue lost on switch

Solution: use Redis for sessions (shared, not per-server)

Kubernetes:

Two Deployments (blue and green)

Service selector switches between them

kubectl patch service myapp -p '{"spec":{"selector":{"version":"green"}}}'

Database schema migration for Blue-Green:

Step 1: Add new column (backward compatible)

Step 2: Deploy new version using new column

Step 3: Backfill data

Step 4: Remove old column (after old version retired)

= Expand-Contract pattern

Interview tip: "Blue-Green requires backward-compatible DB schemas during the window when both versions run. Expand-contract (Parallel Change) pattern solves this. Know it."

Q72. How do you design for high availability?

High
availability

Medium

High Availability: system continues operating despite failures.

Target: 99.99% (52 min downtime/year) or better.

Key principles:

- Eliminate single points of failure (SPOF)
- Redundancy at every tier
- Health checks + automatic failover
- Multi-AZ / multi-region deployment
- Stateless services (any instance can handle any request)
- Graceful degradation (partial functionality, not complete outage)

Layer by layer:

- DNS: multi-AZ Route53 with health checks
- Load Balancer: multiple LB instances (AWS ALB is managed/auto-HA)
- Application: multiple instances across AZs, auto-scaling
- Cache: Redis Sentinel or Cluster (automatic failover)
- Database: primary + replica in different AZs, automated failover (RDS Multi-AZ)
- Storage: S3 (11 nines durability), replicated across AZs

Active-Active vs Active-Passive:

- Active-Active: all nodes serve traffic simultaneously
 - Better utilisation, immediate failover
 - Requires conflict resolution for writes
- Active-Passive: one node serves traffic, other on standby
 - Simpler, no conflict resolution
 - Failover time = detection + switchover (~60s)

RTO and RPO:

- RTO (Recovery Time Objective): maximum acceptable downtime
- RPO (Recovery Point Objective): maximum acceptable data loss
- "RTO=1h, RPO=1min" means: back up within 1 hour, lose max 1 min of data

Multi-region:

- Active-Active: Route53 Latency routing, both regions serve traffic
- Active-Passive: Route53 Failover routing, secondary on standby

Interview tip: "State RPO and RTO for your design. 'This design has RTO=30s (Aurora failover time) and RPO=0 (synchronous replication).' Numbers make your answer concrete."

Q73. What is graceful degradation?

Resilience

Medium

Graceful degradation: when components fail, serve degraded but functional experience.

vs Hard failure: complete service outage.

Examples:

- Netflix: recommendations service down → show generic popular titles, not error page
- Twitter: timeline aggregation slow → show cached timeline, not spinner
- E-commerce: inventory service down → show product, remove "in stock" indicator
- Search: ML ranking service down → fall back to simple chronological results

Implementation patterns:

- Fallback cache: serve stale data from cache if primary source fails

Default value: if personalisation fails, serve default content
 Feature flag disable: turn off non-essential features under load
 Queue and retry: accept request, process later when service recovers
 Static fallback: serve pre-rendered static page if dynamic fails

Circuit breaker for graceful degradation:

```
func getRecommendations(userID string) []Product {
    result, err := circuitBreaker.Execute(func() ([]Product, error) {
        return recommendationService.Get(userID)
    })
    if err != nil {
        return popularItems.Get() // fallback: popular items
    }
    return result
}
```

Timeout hierarchy:

User-facing API timeout: 1s
 Downstream service timeout: 500ms (gives time for fallback)
 DB query timeout: 200ms

SLO for degraded mode:

Normal: 99.9% of requests < 100ms
 Degraded (with fallback): 99.9% of requests < 200ms (but no outage)

Interview tip: "Graceful degradation shows operational maturity. For any design, ask: 'What happens when component X fails? What does the user see?' Good systems degrade gracefully, not catastrophically."

Classic System Designs

Q74. Design a URL shortener (like bit.ly).

Classic design

Medium

Requirements:

Functional: shorten long URL, redirect to original, analytics
 Non-functional: 100M URLs, low latency redirect (<10ms), high availability

Estimation:

Writes: 1M new URLs/day = 11 QPS
 Reads: 100M redirects/day = 1,160 QPS (100:1 read:write)
 Storage: $1M \times 365 \times 5yr \times 500B = \sim 900$ GB

Short code generation:

Option 1: MD5(longURL) → take first 7 chars (collision risk)
 Option 2: Counter-based → base62(auto-increment_id)
 1 = "1", 62 = "Z", 3.5B = 7 chars in base62
 Option 3: Pre-generate random 7-char codes, store in unused pool

API design:

POST /shorten {longURL} → {shortURL: "https://bit.ly/Ab3cX9Y"}
 GET /{code} → 301/302 redirect to longURL

Data model:

urls table: code VARCHAR(7) PK, long_url TEXT, user_id, created_at, expires_at, clicks

Architecture:

Client → LB → API Servers → [Cache: Redis] → DB (PostgreSQL/DynamoDB)
 Redirect path: check Redis first (sub-ms) → miss → DB → cache for next time

Cache strategy:

Cache hot URLs (top 20% = 80% of traffic)
 Key: short_code, Value: long_url, TTL: 24h
 Cache hit rate: ~80%

Redirect: 301 (permanent, cached by browser) vs 302 (temporary, analytics see every visit)
 For analytics: 302

Analytics:

Async: log click event to Kafka → consumer updates click_count
 Don't synchronously update DB on every redirect (bottleneck)

Custom short codes: allow user to specify → check uniqueness
 Expiration: soft delete via expires_at column

Interview tip: "Interviewers love to ask: '301 vs 302?' Answer: 301 is cached by browser (good for performance, bad for analytics). 302 hits your server every time (good for analytics). Bit.ly uses 302 for analytics."

Q75. Design a rate limiter.

Classic design

Medium

Requirements:

Functional: limit requests per IP, per user, per API key
 Non-functional: low latency (<5ms overhead), distributed (multiple servers)

Rate limiting algorithms:

Token bucket: allow burst, avg rate enforced → most flexible
 Sliding window counter: approximate, low memory → good balance
 Sliding window log: exact, high memory → use for payments only

Architecture:

[Client] → [LB] → [API Gateway with rate limiter] → [Services]
 Rate limiter checks: Redis.INCR(key, window) < limit

Redis sliding window counter:

Key: "ratelimit:{user_id}:{window_start}"
 INCR key → get count
 EXPIRE key TTL
 If count > limit → 429

Rule storage:

Config service or DB: {endpoint: "/checkout", limit: 10, window: "1min"}
 Cached in API Gateway memory, refreshed every 60s

Distributed consideration:

Each API server has local counter (inaccurate with N servers)
 Use Redis: single source of truth → accurate but +1ms latency
 Lua script for atomic INCR + check:
 local count = redis.call('INCR', key)
 if count == 1 then redis.call('EXPIRE', key, ttl) end
 return count

Response headers:

X-RateLimit-Limit: 100
 X-RateLimit-Remaining: 45
 X-RateLimit-Reset: 1609459200
 Retry-After: 60 (on 429)

Scale:

Redis cluster for rate limit keys
 Shard by {user_id} hash for even distribution

Read replicas for high-QPS rate check (but requires async counter → less accurate)

Interview tip: "The Lua script for atomic INCR + EXPIRE is the production implementation. Without atomicity: two goroutines both check limit 99 → both pass → 101 requests allowed. Lua prevents this race."

Q76. Design a key-value store (like Redis).

Classic design

Hard

Requirements:

Get(key), Set(key, value), Delete(key), TTL support
Non-functional: <1ms p99, 1M ops/sec, durable, replication

Single-node architecture:

In-memory hash table: O(1) get/set
WAL (Write-Ahead Log): durability
Background snapshot (RDB): periodic full dump
LRU eviction when memory full

Data structures:

String: raw bytes (key → byte[])
Hash: key → hash map
List: key → doubly-linked list
Sorted Set: key → skip list + hash map
(see Redis internals)

Persistence:

AOF (Append-Only File): log every command → replay on restart
Durable: lose at most last fsync window (1s by default)
RDB (snapshot): periodic dump → fast startup, may lose minutes of data
Both: RDB for fast startup + AOF for durability (Redis default in production)

Replication:

Primary handles writes + reads (or separate replicas for reads)
Replicas receive WAL stream from primary
Eventual consistency: replication lag possible
Sentinel: automatic failover on primary death

Cluster (horizontal scale):

16,384 hash slots across N nodes
Each node owns a range of slots
Client-side routing: client gets MOVED redirect if wrong node

Memory optimization:

Compressed encoding for small objects (ziplist, intset)
Shared integers (0-9999 preallocated)
LZF compression for large strings

Interview tip: "Don't reinvent Redis in the interview — explain its design. Focus on: in-memory hash table, AOF+RDB persistence, primary-replica replication, Sentinel for HA. These are the key design decisions."

Q77. Design a notification system.

Classic design

Medium

Requirements:

Send push, email, SMS, in-app notifications
Support: immediate, scheduled, user preferences
Non-functional: 1M notifications/day, low latency, high deliverability

Estimation:

1M notif/day = ~12 QPS average, 100 QPS peak
Fanout example: 100K users following celebrity → 100K push notifications

Architecture:

```

Client → API → Notification Service → Kafka
                                   → Email worker → SendGrid/SES
                                   → Push worker → FCM/APNs
                                   → SMS worker → Twilio

```

Data model:

```

notifications: (id, user_id, type, template_id, payload, scheduled_at, sent_at, status)
user_preferences: (user_id, channel, enabled, quiet_hours_start, quiet_hours_end)
devices: (user_id, device_token, platform, created_at)

```

User preferences check:

```

Is notification enabled for this channel?
Is user in quiet hours? (if low priority, defer)
Rate limiting: max 5 push per hour

```

Template system:

```

Templates stored in DB: "Hello {{name}}, your order {{order_id}} shipped!"
Render at send time with actual data
Supports i18n (language per user preference)

```

Delivery guarantees:

```

Kafka for durability: message not lost even if worker crashes
Retry: failed delivery retried 3× with backoff
DLQ: after 3 failures → dead letter queue → alert

```

Tracking:

```

Delivered, Opened, Clicked events
Webhook from FCM/APNs on delivery/failure
Store in ClickHouse for analytics (not PostgreSQL — high volume)

```

Priority queues:

```

High priority (OTP, password reset): separate fast queue
Low priority (marketing): bulk queue, send during off-peak

```

Interview tip: "Template + user preferences + delivery guarantees = full notification system. Highlight the priority queue for OTPs — users won't wait 5 minutes for a one-time password."

Q78. Design Twitter's timeline.

Classic design

Hard

Requirements:

```

Post tweet, follow/unfollow, view home timeline (reverse chron)
Non-functional: 300M DAU, 500M tweets/day, <500ms timeline load

```

Timeline approaches:**Fan-out on write (push model):**

```

When Alice tweets → precompute timeline for all followers
Store tweet_id in each follower's timeline cache (Redis list)
On read: fetch list of tweet_ids → batch fetch tweets → render
Pros: read is fast (pre-computed)
Cons: celebrities with 100M followers → 100M writes per tweet

```

Fan-out on read (pull model):

```

When user opens timeline → fetch all followees' tweets → merge sort
Pros: no write amplification
Cons: slow for users with many followees (thousands of followees)

```

Twitter's hybrid approach:

```

Normal users: fan-out on write (most users have <10K followers)

```

Celebrities (>1M followers): fan-out on read
 On timeline read: merge pre-computed timeline (cache) + celebrity tweets

Data model:

tweets: (tweet_id, user_id, content, created_at) [tweet snowflake ID encodes time]
 followers: (follower_id, followed_id)
 timeline_cache: Redis sorted set per user (tweet_id → timestamp)

Timeline read:

ZREVRANGE timeline:{user_id} 0 19 → 20 tweet IDs
 Batch fetch from tweets table/cache
 Inject celebrity tweets → merge by timestamp
 Return

Write path:

POST /tweet → write to tweets table → enqueue fan-out job
 Fan-out worker: get followers → for each → ZADD timeline:{follower_id} timestamp tweet_id

Cache:

timeline_cache in Redis: last 1000 tweets per user
 tweet details in Memcached (tweet_id → tweet object)

Scale:

Tweets: write to Cassandra (time-series, write-heavy)
 Timeline: Redis cluster (sorted sets per user)
 Search: Elasticsearch

Interview tip: "The fan-out hybrid model (write for regular, read for celebrities) is the key insight Twitter uses. Know it verbatim — it's asked constantly."

Q79. Design a distributed file storage system (like S3).

Classic design

Hard

Requirements:

Upload/download files, list files, delete
 Non-functional: 1 billion files, 100 TB, 99.999999999% (11 nines) durability

Architecture:

Client → LB → API Gateway
 API Gateway → Metadata Service (which chunk → which server)
 → Chunk Servers (actual data storage)

Chunking:

Split file into 64MB chunks
 Each chunk replicated 3× across 3 chunk servers
 Checksum per chunk (MD5/SHA256) for integrity verification

Metadata Service (Zookeeper + DB):

file_id → {file_name, size, owner, chunks: [chunk_id, ...]}
 chunk_id → {servers: [server1, server2, server3], checksum}

Chunk server:

Stores chunks as local files on disk
 Heartbeat to metadata service every 10s
 GC: remove orphan chunks (no reference in metadata)

Upload flow:

Client → API → Metadata: allocate chunk IDs + servers
 Client → Chunk servers: upload chunks in parallel
 Metadata: mark file complete

Download flow:

Client → Metadata: get chunk_id → server mappings
 Client → Chunk servers: download chunks in parallel
 Client: reassemble + verify checksums

Replication strategy:

Cross-AZ: chunks on servers in different AZs
 Rack-aware: different racks within same AZ
 Repair: background scanner verifies checksums → re-replicates missing chunks

S3 uses Reed-Solomon erasure coding:

vs 3× replication: 6-of-9 coding uses 1.5× storage vs 3× storage
 Rebuild from any 6 of 9 shards

Interview tip: "Durability = checksums + replication + background integrity scanning. '11 nines durability' = if you stored 10 billion files for 10,000 years, you'd expect to lose one. Reed-Solomon is how S3 achieves it."

Q80. Design a search engine (like Elasticsearch/Google Search).

Classic design

Hard

Requirements:

Index documents, full-text search, relevance ranking
 Non-functional: 1B documents, sub-100ms query, near-real-time indexing

Core components:

1. Document Store:

Stores raw documents: {doc_id, url, title, body, metadata}
 DynamoDB or Cassandra (key = doc_id, read by doc_id)

2. Inverted Index (heart of search engine):

term → posting list [{doc_id, TF, positions}, ...]
 TF = term frequency in document
 Built from tokenized, normalized text (lowercase, stem, stop words removed)
 Stored on disk in immutable segments (SSTable-like)

3. Indexing Pipeline:

Document → Tokenizer → Stemmer → Stop word filter → Inverted index update
 Batch indexing: MapReduce over all documents
 Incremental: new documents → Kafka → indexing service

4. Query Processing:

Query tokenized → look up each term → intersect posting lists (AND)
 Scoring: BM25 (modern TF-IDF variant)
 $\text{score} = \text{IDF} \times \text{TF} \times \text{field length normalisation}$
 Rank by score → return top K

5. Distributed Search:

Sharding: documents distributed across N shards (by doc_id hash)
 Query: broadcast to all shards → each returns top K → merge sort → global top K
 Replication: each shard replicated 3×

6. Caching:

Popular queries cached in Redis (query → results)
 TTL: 5 minutes (results may change)

Elasticsearch architecture:

Index → Shards → Segments (Lucene)
 Each shard = Lucene index (inverted index + document store)
 Refresh interval: 1s (near real-time)

Interview tip: "Inverted index is the core data structure. Know: term → posting list. Know BM25 scoring. Know sharded search (scatter-gather pattern). These three cover 80% of search engine questions."

Q81. Design a ride-sharing system (like Uber).

Classic design

Hard

Requirements:

Rider requests ride, match with nearby driver, real-time tracking
Non-functional: 1M concurrent rides, <2s matching, GPS update every 5s

Estimation:

1M active drivers, each updates location every 5s
Location write QPS: $1M / 5 = 200,000$ writes/sec

Architecture:

Rider App → LB → [API Gateway] →
 Location Service (GPS updates from drivers)
 Matching Service (find nearby driver)
 Trip Service (manage active trips)
 Payment Service (charge on trip complete)
 Notification (push to rider/driver)

Location Service:

Driver app → POST /location {lat, lng, timestamp}
 Store in Redis geospatial (GEOADD driver:available lat lng driver_id)
 GEOSEARCH: find drivers within 2km of rider's location
 Redis GEODIST, GEOSEARCH: $O(N+\log(M))$ where M=elements in radius

Geospatial indexing:

Redis GEOSEARCH (recommended):
 GEOSEARCH drivers:available FROMLONLAT lng lat BYRADIUS 2 km
 Google S2 Library (Uber's approach):
 Earth divided into cells at different levels
 Level 12 cell $\approx 2\text{km}^2$ → hash driver into cells
 Find drivers: look up rider's cell + neighboring cells

Matching algorithm:

Get available drivers in 2km radius
 Score: distance + driver rating + ETA
 Assign best driver (lock in DB to prevent double-assignment)
 Race condition: two riders match same driver → use DB transaction/lock

Trip management:

Trip states: requested → accepted → arriving → in_progress → completed
 Each state change → event to Kafka → notification service

Real-time tracking:

WebSocket connection between driver app and rider app (via server)
 Driver → server → Kafka → push to rider's WebSocket
 Map updates every 5s with driver's GPS

Surge pricing:

demand = ride requests in area / supply = available drivers
 surge_multiplier = $f(\text{demand} / \text{supply})$
 Recalculate every minute per geo-cell

Interview tip: "Geospatial indexing is the core challenge. Redis GEOSEARCH or S2 library are the two main approaches. Uber uses H3 (hexagonal hierarchy) for their zone system. Know at least one approach deeply."

Q82. Design a video streaming platform (like YouTube).

Classic design

Hard

Requirements:

Upload video, stream video, search, recommendations

Non-functional: 500M DAU, 500 hours of video uploaded/minute, 1B hours watched/day

Upload pipeline:

1. User uploads raw video → Object store (S3)
2. Upload job triggers transcoding pipeline (Kafka event)
3. Transcoding service: convert to multiple resolutions (360p, 720p, 1080p, 4K)
4. Generate thumbnail
5. Update metadata DB: video available
6. CDN: pull-on-first-access for each PoP

Transcoding:

Expensive: 1 hour of video → 30 min transcoding (parallel)

AWS Elastic Transcoder / FFmpeg clusters

Adaptive Bitrate (ABR): HLS/DASH

Multiple quality levels, player switches based on bandwidth

Segment size: 2-10 seconds per segment

Streaming (adaptive bitrate):

Client: requests video → gets playlist (m3u8)

Playlist: list of segment URLs at different qualities

Client: monitors download speed → picks best quality

Segment: 5-10 second chunks → switch quality every segment

Data model:

videos: (video_id, uploader_id, title, description, duration, status, views)

video_files: (video_id, resolution, file_url, size)

comments: (video_id, user_id, text, created_at)

view_counts: ClickHouse (high write volume, analytics)

CDN strategy:

Popular videos: push to all CDN PoPs proactively

Long tail: pull CDN (fetch from origin on first request per PoP)

~20% of videos = ~80% of views (cache those aggressively)

Recommendation:

Collaborative filtering: users who watched A also watched B

Watch history + search history → embedding model → similarity

Pre-computed recommendations updated hourly (MapReduce / Spark)

View counter:

INCR in Redis → async batch write to DB every 5 minutes

Avoid DB write on every view (bottleneck at 1M QPS)

Interview tip: "Adaptive Bitrate (HLS/DASH) is the key YouTube/Netflix innovation. Without it: fixed quality → buffering on slow connections. With ABR: smooth experience regardless of bandwidth."

Q83. Design a social network feed (like Facebook News Feed).

Classic design

Hard

Requirements:

Post content, follow people, home feed (ranked)

Non-functional: 1B users, 500M posts/day, <200ms feed load

Data model:

users: (user_id, name, email)

posts: (post_id, user_id, content, media_urls, created_at)

```
follows: (follower_id, followed_id)
feed: (user_id, post_id, score, created_at) [pre-computed]
```

Feed generation approaches:

Fan-out on write (push):

Alice posts → fanout worker → writes to feed table for all Alice's followers
 Pros: read is $O(1)$ `SELECT * FROM feed WHERE user_id=? ORDER BY score DESC`
 Cons: Alice has 100M followers → 100M writes (celebrity problem)

Fan-out on read (pull):

Read time: `SELECT * FROM posts WHERE user_id IN (followees) ORDER BY created_at DESC`
 Pros: no write amplification
 Cons: large followee lists → slow (join across millions of posts)

Facebook's hybrid:

Normal users: fan-out on write (push)
 Celebrities (>10K followers): fan-out on read
 At read time: merge pre-computed feed + celebrity posts

Feed ranking:

Score: recency + engagement (likes, comments, shares) + author relationship
 EdgeRank (Facebook's original): $\text{Affinity} \times \text{Weight} \times \text{Time decay}$
 Modern: deep learning model (trained on click-through rate)
 Re-rank on every feed request (computationally expensive)
 Pre-score: compute rough scores on write, fine-tune on read

Caching:

Top 1000 feed items per user in Redis list
 Older posts: paginate from feed table in DB

Pagination:

Cursor-based: `?after=post_id_xyz` (not offset-based)
 New posts inserted at top → offsets shift → cursor stable

Interview tip: "Facebook's EdgeRank is worth mentioning by name. The hybrid fan-out approach (push for normal, pull for celebrities) is the same as Twitter's. Both companies converged on this solution independently."

Q84. Design a chat application (like WhatsApp).

Classic design

Hard

Requirements:

1:1 and group messaging, message delivery status, offline messages
 Non-functional: 1B users, 100B messages/day, <100ms delivery

Estimation:

$100B \text{ messages/day} = 1.16M \text{ messages/sec}$

Architecture:

Client $\leftrightarrow$ WebSocket $\leftrightarrow$ Chat Server $\leftrightarrow$ Kafka $\leftrightarrow$ Chat Server $\leftrightarrow$ WebSocket $\leftrightarrow$ Client

Chat Server: maintains WebSocket connections
 Message DB: store messages
 User Service: manage user presence/status
 Notification: push when recipient offline

Message flow (A → B):

1. Client A → WebSocket → Chat Server A
2. Chat Server A → write to Message DB + Kafka
3. If B online: Kafka consumer on Chat Server B → push via WebSocket to B
4. If B offline: Push notification (FCM/APNs)

5. B connects → pulls missed messages from Message DB

Message delivery status:

- Sent (1 tick): server received message
- Delivered (2 ticks): recipient's device received
- Read (2 blue ticks): recipient opened conversation
- Implemented via ACK messages from client

Data model:

```
messages: (message_id, chat_id, sender_id, content, type, created_at, deleted_at)
chat_members: (chat_id, user_id, joined_at, last_read_at)
Cassandra: time-series, high write, queries by (chat_id, time range)
message_id: Snowflake ID (time-ordered, globally unique)
```

Group messages:

```
Group → N members
Option 1: fan-out on write (write to each member's inbox) → simple but N writes
Option 2: store once, members read from group chat → efficient storage
WhatsApp: store once + fan-out for active members' caches
```

End-to-end encryption:

Signal protocol: each message encrypted with recipient's public key
Server never sees plaintext (can't comply with law enforcement easily)

WebSocket management:

```
Sticky sessions: consistent hash to same chat server per user_id
On reconnect: client sends last_message_id → server sends missed messages
```

Interview tip: "WebSocket for real-time + Kafka for fan-out + Cassandra for message storage is the production stack. Signal protocol for E2E encryption — mention it even if you don't implement it."

Q85. Design a payment processing system.

Classic design

Hard

Requirements:

Charge card, refund, view transaction history
Non-functional: 99.999% availability, exactly-once processing, fraud detection

Key constraints:

```
Idempotency: network retries must not double-charge
ACID: transaction must be atomic (debit + credit)
Compliance: PCI-DSS (don't store raw card numbers)
Audit: every operation logged
```

Architecture:

[illegible]

Idempotency:

```
Client sends Idempotency-Key: UUID with every payment request
Server: INSERT INTO idempotency_keys ... ON CONFLICT DO NOTHING
Same key → return stored response (no double charge)
```

Charge flow:

1. Client → POST /payments {amount, card_token, idempotency_key}
2. Check idempotency key (already processed?)
3. Fraud check (ML model: amount, merchant, device fingerprint)
4. Call PSP (Stripe): charge card token
5. On success: INSERT INTO ledger (amount, type=DEBIT, user_id, created_at)
6. Return success response + store in idempotency table

Double-entry bookkeeping:

Every transaction: one DEBIT + one CREDIT (must balance)
 Ledger is append-only (never update/delete)
 Balance = SUM of all entries for account

Failure handling:

PSP timeout: was the charge made? Check PSP status API (idempotent GET)
 If unknown: return 202 Accepted → async check → notify via webhook

Outbox pattern:

Write to payment_events outbox in same DB transaction as ledger
 Kafka publisher reads outbox → publishes PaymentProcessed event
 Downstream: email, analytics, fraud update – all async

PCI-DSS compliance:

Never log raw card numbers
 Use tokenization: card stored at Stripe, you store token only
 Network segregation: payment service in separate VPC

Interview tip: "Idempotency + Outbox pattern + double-entry bookkeeping are the three payment system pillars. Mentioning PCI-DSS and tokenization shows operational awareness."

Q86. Design a distributed job scheduler (like Airflow/Cron).

Classic design

Hard

Requirements:

Schedule jobs (cron or one-shot), execute, handle retries
 Non-functional: 10K jobs, at-least-once, exactly-once with idempotent jobs

Components:

Scheduler: reads job definitions, triggers at right time
 Job Store: DB of job definitions + execution history
 Executor: executes jobs, reports status
 Worker Pool: actual workers that run job code
 Leader Election: only one scheduler instance runs at a time

Data model:

jobs: (job_id, name, cron_expr, handler, payload, max_retries, timeout, enabled)
 job_runs: (run_id, job_id, scheduled_at, started_at, completed_at, status, error)

Scheduler loop (runs on leader only):

every second:
 jobs_due = SELECT * FROM jobs WHERE next_run_at <= NOW() AND enabled=true
 for each job: create job_run, enqueue to worker queue

Leader election:

Only one scheduler instance should run (prevent duplicate job runs)
 Use DB SELECT ... FOR UPDATE or Redis SETNX
 Renew every 30s; other schedulers wait

Worker execution:

Worker picks job from queue (Kafka or DB-backed queue)
 Execute job handler
 On success: UPDATE job_runs SET status='success', completed_at=NOW()
 On failure: check retry count → retry (exponential backoff) or mark failed

Exactly-once semantics:

Check run_id in idempotency table before executing
 Claim job: UPDATE job_runs SET status='running', worker_id=? WHERE status='pending'
 If UPDATE affects 0 rows → another worker claimed it → skip

Distributed execution:

Workers can run on different machines
Job handler is a function registered by name
Worker fetches code or uses pre-deployed job containers

Timezone handling:

Store cron in UTC
Convert to user's timezone for display only
DST transitions: skip or repeat? (run once, usually)

Monitoring:

Job success rate dashboard
Stalled jobs alert: status='running' for > 2× timeout → mark as failed

Interview tip: "Leader election for the scheduler prevents duplicate executions. The atomic UPDATE ... WHERE status='pending' prevents two workers running the same job. These two together give at-most-once execution."

Q87. Design a hotel/flight booking system.

Classic design

Hard

Requirements:

Search available rooms/flights, book, cancel, payment
Non-functional: no double booking, high availability during peak (holidays)

Double-booking prevention:

Critical: two users must not book the same seat/room simultaneously

Option 1: SELECT FOR UPDATE (pessimistic lock)

```
BEGIN
SELECT * FROM seats WHERE id=? AND status='available' FOR UPDATE
UPDATE seats SET status='booked', user_id=? WHERE id=?
COMMIT
Pro: safe; Con: high contention during peak
```

Option 2: Optimistic locking

```
SELECT seat, version FROM seats WHERE id=?
UPDATE seats SET status='booked' WHERE id=? AND version=?
If 0 rows updated → conflict, retry or fail
```

Option 3: Compare-and-swap at DB level

```
UPDATE seats SET status='booked', user_id=?
WHERE id=? AND status='available'
Check affected_rows == 1
```

Architecture:

Client → API → Inventory Service → DB (seats/rooms)
→ Payment Service (charge on success)
→ Notification (confirmation email)
Two-phase booking: hold → confirm/release (like airline seat hold)

Seat hold:

User selects seat → Hold for 10 minutes (soft lock)
If user pays → confirm hold → permanent booking
If 10 min expires → release hold → seat available again

Search optimization:

Pre-computed availability: cache available seat counts per flight
Real-time: DB for exact availability check at booking time
Elasticsearch for flexible search (date range, price range, destination)

Payment:

Charge AFTER hold confirmed

Release hold if payment fails
 Outbox pattern: booking_confirmed → email/notification

Scale during peak:

Queue requests during spikes (Black Friday, Cyber Monday)
 Show "virtual queue" to user
 Process at steady rate

Interview tip: "The seat hold (two-phase) pattern is how airlines work. No airline actually charges you before confirming the seat. The SELECT FOR UPDATE + timeout is the core double-booking prevention."

Advanced System Designs

Q88. Design a distributed cache system (like Memcached cluster).

Advanced
design

Hard

Requirements:

Get, Set, Delete, TTL, distributed across N nodes
 Non-functional: 1M QPS, <1ms p99, horizontal scale

Client-side sharding (consistent hashing):

Client maps key → node (no server-side routing)
 libmemcached: consistent hashing built-in
 Adding node: ~1/N keys remapped (vs naive modulo: all keys remapped)

Node architecture:

In-memory hash table (slab allocator — avoids fragmentation)
 LRU eviction per slab class
 Single-threaded (like Redis) or multi-threaded (Memcached: multi)

Slab allocator:

Memory divided into slab classes (64B, 128B, 256B, ...)
 Each slab class: array of fixed-size chunks
 No fragmentation (allocate whole chunk, not arbitrary size)
 Trade-off: waste if item << chunk size

Facebook's Memcache paper insights:

Get/Set over UDP (faster, no connection overhead, best-effort)
 TCP only for writes to primary (reliability)
 Consistent hashing: minimal remapping on scale

Thundering herd prevention:

"stale-while-revalidate" mode: serve stale, refresh asynchronously
 Lease system: only ONE client fetches from DB on cache miss
 First client gets a "lease" (token); others back off and wait
 If client fails, lease expires → next client can try

Replication:

Memcached: no built-in replication → client writes to multiple pools
 For HA: write to pool A and pool B in parallel
 Read: pool A only (replicated for DR, not primary HA)

Regional vs local caches:

Regional: shared cache across all app servers in region
 Local: in-process cache (faster but not shared, inconsistent)
 Both: local L1 (1-10K items, sub-ms) + regional L2 (100M items, 1ms)

Interview tip: "Facebook's Memcache paper (2013) is required reading for distributed caching. The lease system for thundering herd prevention is their key contribution. Mention it."

Q89. Design a real-time leaderboard.

Advanced
design

Medium

Requirements:

Update score, get rank, get top N, get neighbors of user
Non-functional: 10M players, real-time updates, <10ms queries

Redis Sorted Set (core data structure):

ZADD leaderboard {score} {player_id} $O(\log N)$ insert/update
ZRANK leaderboard {player_id} $O(\log N)$ rank lookup
ZREVRANGE leaderboard 0 99 $O(\log N + 100)$ top 100
ZSCORE leaderboard {player_id} $O(1)$ score lookup
ZRANGEBYSCORE leaderboard min max $O(\log N + M)$ range

Player's neighbors (nearby ranks):

rank = ZREVRANK leaderboard player_id
ZREVRANGE leaderboard (rank-5) (rank+5) WITHSCORES → neighbors

Update flow:

User scores points → ZADD leaderboard new_score player_id
Atomic: ZADD updates if score already exists

Multiple leaderboards:

"leaderboard:global"
"leaderboard:weekly:2024-W42"
"leaderboard:country:IN"
Separate Redis Sorted Sets

Time-bucketed leaderboards:

Weekly leaderboard: keys expire after 1 week
Redis ZADD leaderboard:weekly:{week} score player_id
TTL: EXPIREAT leaderboard:weekly:{week} {end_of_week_timestamp}

Large scale (>100M players):

Redis cluster: shard by player_id range or hash
But global rank requires merge across shards = complex
Alternative: approximate rank (good enough for most games)
Count players with score > mine = rank
Possible with segment tree or skip list approximation

Tie-breaking:

Same score: earlier achievers ranked higher
Compound score: $\text{actual_score} * 10^9 + (\text{MAX_TIME} - \text{timestamp})$
Monotonically decreasing → earlier time = higher rank

Interview tip: "Redis Sorted Set is the canonical answer for leaderboards. Enumerate all the operations with their time complexity. ZREVRANK in $O(\log N)$ is what makes it suitable for real-time use."

Q90. Design a web crawler.

Advanced
design

Hard

Requirements:

Crawl the web, extract URLs, store content, respect robots.txt
Non-functional: 1B pages, politeness, deduplication

Estimation:

1B pages, avg 100KB each = 100 TB

Rate: 100M pages/day = 1,160 pages/sec

Architecture:

URL Frontier → Fetcher → Parser → Link Extractor → Deduplicator → URL Frontier
→ Content Store

Components:

URL Frontier (priority queue):

Stores URLs to crawl
Priority: PageRank, freshness, importance
Politeness: one queue per domain, delay between requests
Redis sorted set per domain: (URL → priority)

Fetcher:

HTTP client pool (100K concurrent connections)
Respect robots.txt (cache per domain)
Respect Crawl-delay in robots.txt
DNS cache (don't resolve same host repeatedly)

Parser:

HTML → extract links + text content
Normalize URLs (lowercase, remove fragment, canonical form)

Deduplication:

Bloom filter for visited URLs (avoid re-crawling)
SimHash for near-duplicate content detection (same content, different URL)

Storage:

Raw HTML: object store (S3)
Parsed text + metadata: Elasticsearch
URL graph: graph DB (for PageRank computation)

Scale-out:

Multiple fetcher workers, partitioned by domain
Consistent hash: domain → worker (politeness: only one worker per domain)

Re-crawl strategy:

Freshness: high-value pages re-crawled daily (news), others monthly
Change detection: check Last-Modified header or hash content

Interview tip: "The URL Frontier with per-domain queues + politeness delay is the core design challenge. Without politeness, you'll get IP-banned by every site. Bloom filter for dedup prevents re-crawling 1B visited URLs."

Q91. Design a stock trading system.

Advanced
design

Hard

Requirements:

Place orders, match buy/sell orders, track portfolio, market data
Non-functional: low latency (<1ms order matching), strict ordering, no data loss

Key concepts:

Order book: sorted list of all open buy/sell orders per instrument
Matching engine: matches buy orders against sell orders
Market order: execute at best available price
Limit order: execute at specified price or better

Order book structure:

Bids (buyers): sorted descending by price
Asks (sellers): sorted ascending by price

Spread: ask - bid (tighter = more liquid)

Matching engine (critical path – ultra low latency):

Single-threaded (no locks!) for each trading pair
 In-memory order book (hash map + sorted structure)
 Persist to WAL before ACK
 Match: highest bid $\geq$ lowest ask $\rightarrow$ trade

Order{type:LIMIT, side:BUY, price:100, qty:10}

Ask book has 100@ ask price $\rightarrow$ MATCH $\rightarrow$ trade

Data structures for order book:

Price levels: sorted map (price $\rightarrow$ total qty)
 Orders at price: queue (FIFO for fairness)
 Lookup by order_id: hash map

LMAX Disruptor pattern (used by trading firms):

Lock-free ring buffer for inter-thread communication
 Single-writer principle
 Cache-line padding (prevent false sharing)

Architecture:

Client $\rightarrow$ OMS (Order Management System) $\rightarrow$ Matching Engine
 Matching Engine $\rightarrow$ Trade events $\rightarrow$ Kafka $\rightarrow$ Settlement, Risk, Market Data

Market data distribution:

Trades and order book updates $\rightarrow$ Kafka $\rightarrow$ Market Data Service
 WebSocket feed to clients (sub-100ms)
 Co-location: low latency clients physically locate servers near exchange

Risk checks (pre-trade):

Check buying power (available cash)
 Position limits (can't hold $>$ X% of stock)
 Fat finger checks (order size within 10% of current market price)

Interview tip: "The matching engine must be single-threaded (no locks = predictable low latency). LMAX Disruptor is the production pattern. WAL ensures no order is lost. These three concepts cover the core of trading system design."

Q92. Design a fraud detection system.

Advanced
design

Hard

Requirements:

Detect fraudulent transactions in real-time ($<$ 100ms), block or flag
 Non-functional: 100K TPS, low false positive rate, adaptive (learns from new patterns)

Architecture (two modes):

Synchronous (real-time): block transaction if fraudulent
 Asynchronous (batch): detect patterns, update models

Real-time path ($<$ 100ms):

Transaction $\rightarrow$ Feature extraction $\rightarrow$ Rules engine $\rightarrow$ ML scoring $\rightarrow$ Decision
 All steps in-process, in-memory where possible

Feature extraction:

Velocity features: 5 transactions in last 10 minutes? (Redis counter)
 Location features: same card used in NY and London within 1 hour?
 Behavioral: amount deviates from user's historical average?
 Device: new device + new location + high amount = suspicious

Rules engine:

Fast explicit rules (no ML needed):

Rule 1: if amount > 10,000 AND new_device → flag

Rule 2: if country != user.home_country AND amount > 500 → flag

Rule 3: velocity > 5 transactions in 60s → block

Rules stored in DB, loaded on startup, hot-reloaded periodically

ML model:

Gradient boosted trees or neural network

Features: transaction features + user history features

Output: fraud probability (0-1)

Threshold: >0.7 → decline, 0.3-0.7 → 3D Secure challenge, <0.3 → allow

Model served via feature store + model server (Feast + Seldon)

Feedback loop:

Customer disputes → labeled fraud → retrain model

Chargeback data → ground truth

A/B test new models: 5% traffic to new model, compare performance

Redis for velocity checks:

INCR "velocity:{card_id}:{minute_bucket}"

EXPIRE 300 (5 minutes)

Atomic: read count + compare in Lua script

Graph analysis (async):

Find fraud rings: device → multiple accounts → same merchant

Graph DB: Neo4j or TigerGraph for entity relationship queries

Interview tip: "The synchronous + asynchronous dual-mode architecture is key. Real-time rules + ML for blocking, batch graph analysis for detecting rings. Feedback loop (disputes → retraining) shows systems thinking."

Q93. Design Google Docs (collaborative editing).

Advanced
design

Hard

Requirements:

Real-time collaborative editing, offline support, conflict resolution

Non-functional: <100ms sync latency, supports 100 concurrent editors per doc

Conflict resolution approaches:

Operational Transformation (OT) – Google Docs:

Each operation transformed against concurrent operations

If Alice inserts "X" at pos 5 and Bob inserts "Y" at pos 5:

Alice's insert is transformed to pos 6 for Bob's view (or vice versa)

Centralized server applies operations in order → source of truth

Server sends back transformed operations to all clients

CRDT (Conflict-free Replicated Data Types) – Figma, Notion:

Operations are inherently commutative (order doesn't matter)

No central coordination needed

Types: CRDT-text (RGA, LSEQ), CRDT-counter, CRDT-set

Better for offline-first / P2P, harder to implement efficiently

Architecture (OT-based like Google Docs):

Client → WebSocket → Doc Server → Redis (op queue) → DB (persisted ops)

Client:

Local buffer: apply operation immediately (optimistic UI)

Send to server: (op, base_revision)

Receive: (transformed_op, new_revision) → apply to local doc

Server:

Serialize all operations (single thread per document)
 Transform incoming op against ops since client's base_revision
 Apply to doc state
 Broadcast transformed op to all other connected clients

Storage:

Full document: periodic snapshot to S3 (every N ops)
 Operation log: Kafka (for replay) + DynamoDB (for queries)
 Reconstruct doc: load last snapshot + replay ops since snapshot

Presence indicators:

Redis Pub/Sub: user cursor positions broadcast to all editors
 Update every 100ms → smooth cursor movement

Offline support:

Buffer ops locally while offline
 On reconnect: send all buffered ops with base_revision
 Server: transform all buffered ops against concurrent ops → merge

Interview tip: "OT vs CRDT is a classic trade-off: OT requires central coordination (simpler for text, server is single source of truth); CRDT is P2P/offline-first (complex to implement correctly). Google chose OT, Figma chose CRDT."

Q94. Design an ads bidding/serving system.

Advanced
design

Hard

Requirements:

Match ads to user context, real-time bidding, click tracking, billing
 Non-functional: 10M ad requests/sec, <10ms ad selection, accurate billing

Real-Time Bidding (RTB) flow:

Publisher page loads → ad request → Ad Exchange
 Ad Exchange → OpenRTB request to all Demand-Side Platforms (DSPs)
 DSPs bid within 100ms → highest bid wins → ad served

Ad serving (direct/owned inventory):

User context: location, device, browsing history, demographics
 Ad selection: candidates → filtering → ranking → serve
 Budget: don't serve ads from depleted campaigns

Targeting:

Audience segments: "25-34, male, interested in tech"
 Contextual: keyword matching on page content
 Behavioral: user visited electronics pages 3× this week
 Retargeting: user added to cart but didn't buy

Ad auction (second-price / Vickrey):

Advertisers bid by CPM (cost per 1000 impressions) or CPC (per click)
 Winner: highest bid, pays second-highest price (+ 1 cent)
 Expected value: clicks × CPC × click-through-rate = expected revenue

Click fraud prevention:

Invalid Traffic (IVT) detection
 Check: same IP clicking many times, bot patterns, impossible click speed
 Machine learning model: probability this click is genuine

Billing:

Impression logged → Kafka → billing consumer → aggregate
 No immediate DB write on every impression (too slow at 10M/sec)
 Batch: aggregate by campaign per minute → write to billing DB
 Idempotency: deduplicate on (impression_id) before charging

Frequency capping:

User shouldn't see same ad >3 times/day

Redis: INCR "freqcap:{user_id}:{ad_id}:{date}" → check < 3

TTL: expire at midnight

A/B testing:

Test ad creative A vs B

Random assignment (consistent hash by user_id)

Measure CTR, conversion rate, revenue per impression

Interview tip: "Second-price auction is Google AdWords' original design. Know why: bidders bid their true value (dominant strategy) because they always pay less than their bid. This shows understanding beyond just the engineering."

Q95. Design an e-commerce platform (like Amazon).

Advanced
design

Hard

Requirements:

Browse products, search, add to cart, checkout, order tracking

Non-functional: 100M DAU, peak 1M QPS (Prime Day), 99.99% availability

Services breakdown:

Product Catalog: browse, search (read-heavy, cacheable)

Inventory: stock levels (write-heavy, consistency critical)

Cart: add/remove items (session, eventually consistent)

Order: place, track, cancel (ACID, event-driven)

Payment: charge, refund (idempotent, audit)

Search: full-text + filters (Elasticsearch)

Recommendation: "also bought" (batch ML, served from cache)

Notification: order updates (email/push, async)

Product catalog:

PostgreSQL for product data

Elasticsearch for full-text search + faceted filters

Cache: Redis/CDN for popular products (read at 1M QPS easily from cache)

Inventory management (hardest part):

Race condition: 1 item left, 1000 users trying to buy

Solution: SELECT ... FOR UPDATE (DB pessimistic lock)

Or: Redis DECR inventory:{product_id} → if < 0, rollback + out-of-stock

Cart:

Redis HASH: {user_id} → {product_id: qty, ...}

TTL: 30 days (persistent for logged-in users)

Merge carts on login (guest → authenticated)

Order state machine:

pending → payment_processing → confirmed → shipped → delivered → cancelled/returned

Flash sale (Peak load):

Cache inventory count in Redis

Pre-sell: accept orders up to cached count

Async: verify actual DB stock, cancel if oversold (compensate)

Separate flash sale service with different limits

Recommendation engine:

"Customers also bought": collaborative filtering on purchase history

Pre-computed hourly with Spark → stored in DynamoDB (product_id → [rec_ids])

Read from cache → sub-millisecond

Search:

Elasticsearch: 1B products, fuzzy search, filters (price, brand, rating)

Autocomplete: separate completion suggester index

Personalized ranking: ML model boosts items user likely to buy

Interview tip: "Inventory management with race conditions is the hardest part. The `SELECT FOR UPDATE` pattern + Redis pre-check is the standard. For Black Friday scale: queue all add-to-cart operations and process sequentially per product."

Real-World Trade-offs

Q96. How do you choose between SQL and NoSQL for a specific use case?

Decision
framework

Medium

Decision framework — ask these questions:

- What are the access patterns?
 "Get user by ID" → key-value lookup → DynamoDB excellent
 "Get all orders for user, joined with product details" → SQL
 "Time-series metrics, always append, range by time" → Cassandra/InfluxDB
- What consistency do you need?
 "User balance" → strong consistency → SQL with ACID
 "User feed" → eventual consistency → DynamoDB
- What is the scale?
 <10M rows/table, <10K QPS → SQL easily handles this
 >1B rows OR >100K QPS → evaluate NoSQL or SQL with sharding
- Does the schema change frequently?
 Stable schema (payments, users) → SQL, schema enforces correctness
 Rapidly changing (user preferences, config) → document DB
- Do you need complex queries?
 Reporting, aggregations, joins → SQL
 Simple get/put by primary key → NoSQL
- What's your team's expertise?
 Most engineers know SQL → lower operational risk
 NoSQL requires tuning knowledge (Cassandra: partition key, replication factor)

Concrete recommendations:

User accounts: PostgreSQL (ACID, joins, mature)
 Product catalog: PostgreSQL + Elasticsearch for search
 Shopping cart: Redis (TTL, fast, acceptable to lose)
 Order history: PostgreSQL (ACID, audit)
 User activity feed: Cassandra (time-series, write-heavy)
 Sessions: Redis (TTL, low latency)
 Analytics events: ClickHouse or BigQuery (columnar, high QPS analytics)
 Relationships: PostgreSQL (FK constraints) or Neo4j (graph traversals)

Interview tip: "Never say 'NoSQL scales better and SQL doesn't.' Say: 'For this access pattern at this scale, DynamoDB is appropriate because X.' Always justify with access pattern + scale + consistency requirements."

Q97. How do you handle eventual consistency in your system?

Consistency
handling

Hard

Accepting eventual consistency requires understanding the user impact and designing for it.

Read-your-writes consistency:

Problem: User updates profile, immediately reads it → sees old value

Solution 1: Route user's reads to primary for 30s after write

HTTP header: X-Consistency: leader / routing table per user

Solution 2: Cache the write locally → serve from local cache for 30s

Solution 3: Read from primary always (sacrifices read scaling)

Monotonic reads:

Problem: Refresh page, see older data than previous request

Cause: routed to different replica with different lag

Solution: sticky sessions → same user always reads from same replica

Implementation: consistent hash(session_id) → replica

Causal consistency:

Problem: Alice replies to Bob's post before Bob sees the original post

Solution: vector clocks or logical timestamps

"If you've seen event X, you'll see all events causally before X"

Handling stale data in UI:

Optimistic updates: immediately update UI on write (assume success)

Rollback: if async confirmation fails, revert UI + notify user

"Your changes have been saved" → stale data risk acceptable

"Your payment was processed" → strong consistency required

Conflict resolution (when multiple writes to same data):

Last Write Wins (LWW): timestamp decides (risk: clock skew)

Multi-value (Dynamo): return all conflicting versions, app resolves

CRDT: mathematically conflict-free merge (Riak, Cosmos DB)

Practical rule:

User-visible state: read-your-writes minimum

Cross-user visible state (feed, counts): eventual consistency OK

Financial state: strong consistency mandatory

Interview tip: "The practical answer: 'We accept eventual consistency for the feed (user sees tweet 200ms later — fine). We require strong consistency for inventory (last item race condition — not fine). Design each component appropriately.'"

Q98. When do you choose microservices vs monolith?

Architecture
decision

Medium

Monolith first (default for new products):**Advantages:**

Simple deployment (one binary)

In-process communication (no network latency)

Easier debugging (single process, single log)

Simpler transactions (ACID across the whole app)

Faster development (no service boundary overhead)

Easier refactoring (all code visible)

When to stay monolith:

Early-stage startup (product-market fit not found)

Small team (<20 engineers)

Low complexity domain

Limited operational expertise

Microservices (when scale demands it):**Advantages:**

Independent deployments (teams ship without coordination)

Independent scaling (scale payment service, not auth service)

Technology heterogeneity (each service picks best tool)

Fault isolation (user service crash doesn't kill payment service)

Team ownership (each team owns one service end-to-end)

When to go microservices:

>50 engineers (coordination overhead justifies it)

Clear bounded contexts (DDD)

Different scaling requirements per component

Need independent release cadences

Mature CI/CD + observability infrastructure

Costs of microservices:

Network latency: function call (0ns) → network call (1-10ms)

Distributed transactions: ACID → Saga + compensations

Operational complexity: N services, N Kubernetes deployments, N alerts

Testing: integration tests across services are complex

Observability: need tracing across services (OpenTelemetry)

Migration path:

Start monolith → identify bottlenecks → extract services one at a time

Strangler Fig pattern: gradual extraction, not big-bang rewrite

Sam Newman's rule: "Don't start with microservices. Start with a monolith.

When the monolith's seams become clear, extract services along those seams."

Interview tip: "The correct answer for an early-stage startup is almost always 'start with a well-structured monolith.' Interviewers are testing whether you blindly jump to microservices or apply judgment. Apply judgment."

Q99. How do you handle database migrations at scale?

Schema
evolution

Hard

Challenge: change DB schema without downtime, while code is deployed.

Rules for zero-downtime migrations:

1. Schema changes must be backward compatible
2. Old code must work with new schema
3. New code must work with old schema (during rolling deploy)

Expand-Contract pattern (Parallel Change):

Step 1: Expand (add, don't remove)

ALTER TABLE users ADD COLUMN display_name VARCHAR(255)

Old code: ignores new column (OK)

New code: writes to new column

Deploy new code

Step 2: Migrate data

UPDATE users SET display_name = name WHERE display_name IS NULL

Run in small batches (avoid locking entire table)

Loop: UPDATE ... WHERE id BETWEEN ? AND ? AND display_name IS NULL

pg_repack: rewrite table without locks (PostgreSQL)

Step 3: Contract (remove old)

After new code is stable (weeks/months later)

ALTER TABLE users DROP COLUMN name

Old code already retired – no dependency

Deploy removal

Operations to avoid during live traffic:

ALTER TABLE ADD COLUMN NOT NULL (table lock in old PG) → add NULL first, then NOT NULL later

DROP TABLE – wrong table → catastrophic

Change column type (implicit full table rewrite) → add new column, migrate, rename

Blue-Green database:

Replication lag + dual-write during migration
 Blue DB (old schema) → new writes to Green DB (new schema) via dual-write
 Verify consistency → cut reads to Green
 Stop dual-write

Tools:

golang-migrate: SQL migration files with up/down
 Flyway, Liquibase: Java-origin, widely used
 Atlas: Go-based, drift detection, cloud-native
 gh-ost (GitHub): online schema change for MySQL (no locks)
 pg_repack: online table reorganization for PostgreSQL

Interview tip: "The expand-contract pattern is the production answer for zero-downtime migrations. Adding NOT NULL columns or changing column types require special handling. Know gh-ost for MySQL or pg_repack for PostgreSQL."

Q100. How do you design a system for 10× current scale?Scalability
planning**Hard**

Framework: identify bottleneck, then apply appropriate technique.

Current state: 10K QPS, 100ms p99

Target: 100K QPS, 100ms p99 (10× scale, same latency)

Step 1: Identify bottlenecks with profiling

Application: CPU? Memory? Thread pool?
 Network: bandwidth? connections?
 Database: QPS limit? Lock contention? Slow queries?
 Cache: hit rate? Evictions?

Step 2: Bottleneck → Solution mapping**CPU-bound application:**

Horizontal scaling (add more servers)
 Auto-scaling based on CPU metrics
 Profile + optimise hot code paths

Database write bottleneck:

Connection pooling (PgBouncer)
 Write batching (batch 100 inserts)
 Async writes (queue → batch writer)
 Sharding (if single-server limit hit)
 CQRS (separate read/write paths)

Database read bottleneck:

Read replicas (distribute read traffic)
 Caching (Redis) for frequent queries
 Query optimisation (EXPLAIN ANALYZE)
 Denormalize (avoid expensive JOINS)
 Search off-load to Elasticsearch

Cache bottleneck:

Redis Cluster (shard cache)
 Increase cache size
 Optimize key design (avoid large keys)

Network bottleneck:

CDN for static assets
 Compression (gzip/brotli)

Connection pooling
HTTP/2 multiplexing

Application state bottleneck:
Stateless services (move state to Redis)
Enables unlimited horizontal scaling

10× doesn't always mean 10× cost:
Caching + CDN: 10× traffic at 2× cost
Algorithmic optimisation: 10× capacity at same cost
Sharding: 10× capacity at 10× cost (last resort)

Priority: optimise → cache → scale out → shard

Interview tip: "The best answer to '10× scale' shows a systematic approach: profile, identify bottleneck, apply targeted solution. Don't blindly add more servers — that only helps CPU-bound services. The database is almost always the bottleneck at scale."

Quick Reference: HLD Decision Cheat Sheet

Decision	Default Choice	Switch When
DB (relational)	PostgreSQL	>1B rows → shard or DynamoDB
DB (document)	MongoDB	Simple key-value → DynamoDB
Cache	Redis	>100GB or >1M ops/s → Redis Cluster
Message queue	Kafka	Simple task queue → SQS/RabbitMQ
Search	Elasticsearch	Simple FTS → PostgreSQL FTS
CDN	CloudFront/Cloudflare	Always use for static assets
Load balancer	AWS ALB	Bare metal → HAProxy/Nginx
Storage	S3	Always for blobs
Service discovery	Kubernetes DNS	Complex routing → Consul
Tracing	OpenTelemetry	Always instrument with OTel

Quick Reference: Number Cheatsheet

Metric	Value
1 day	~86,400 seconds (~100K for estimation)
1M users, 1 req/day	~12 QPS
Single DB server	~10K QPS (simple queries)
Redis	~1M ops/sec (single node)
Kafka	~1M msgs/sec (single broker)
S3 latency	~20-100ms first byte
CDN latency	5-20ms edge hit
SSD sequential read	3-5 GB/s

Network (same AZ)	<1ms
Network (cross-AZ)	1-5ms
Network (cross-region)	50-150ms
3 nines availability	8.77 hours downtime/year
4 nines availability	52.6 min downtime/year

*Good luck with your SDE2 HLD interviews! Draw diagrams, state trade-offs, and always quantify your decisions.

■*

Push to GitHub: `git add . && git commit -m "Add 100 HLD SDE2 interview questions" && git push`